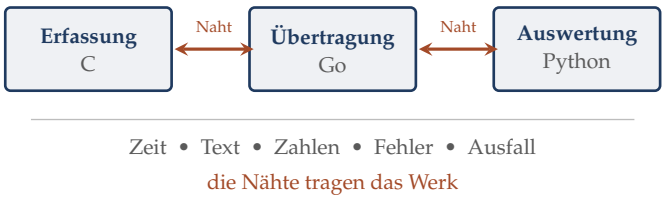
Informatik von Grund auf — Band 23

Vom Programm zum Werk

Sprachübergreifende Entscheidungen



Was bleibt, wenn die Sprache wegfällt
Jeder Schritt erklärt. Mit vollständig entwickelten Programmen.

Stand der Forschung: August 2026

Informatik von Grund auf — Band 23
Vom Programm zum Werk · Programme, die zusammenwirken

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 23: Vom Programm zum Werk*. Fassung 1.0.1. `https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 122 Seiten.

Contents

Vorwort 1

I Das Werk als Ganzes 7

1 Programm und Werk 9

2 Die Naht 15

3 Verträge 21

II Über die Grenze 27

4 Zeit 29

5 Text 35

6 Zahlen 41

III Leistung im Großen 47

7 Messen im Verbund 49

8 Wo die Zeit hingeht 55

9 Zwischenspeicher 61

IV Der Betrieb 67

10 Fehler im Verbund 69

11 Beobachtbarkeit 75

12 Ausfall und Wiederanlauf 81

V Das Urteil **87**

13 Der zweite Entwurf im Großen **89**

14 Wann teilen und wann nicht **95**

15 Der Stand im Jahr 2026 **101**

VI Anhang **107**

A Regeln und Kenngrößen **109**

B Landkarte und Glossar **113**

C Kommentierte Quellen **115**

Vorwort: Die Nähte

Drei Bände haben gezeigt, wie man in einer Sprache gut arbeitet. *Dieser handelt davon, was geschieht, wenn drei solche Teile zusammengefügt werden — und warum die Fehler dann fast alle an den Nähten sitzen.*

Was an der Naht schiefgeht

Frage	Wenn unbeantwortet	
Welcher Zeitpunkt?	**Verläufe falsch**	**Kap. 4**
Welche Zeitzone?	Stunden verschoben	
Welche Kodierung?	**Namen zerstört**	Kap. 5
Welche Genauigkeit?	Stellen erfunden	Kap. 6
Wer wiederholt?	**Werte doppelt**	Kap. 10
Was bei Ausfall?	stiller Verlust	Kap. 12

Sechs Fragen, und keine davon ist eine Sprachfrage. Sie stellen sich zwischen einem C-Programm und einem Go-Programm genauso wie zwischen zwei Python-Modulen — nur fallen sie dort seltener auf, weil beide Seiten dieselben Voreinstellungen mitbringen.

Wo die Fehler tatsächlich sitzen.

Ort	Anteil	
Innerhalb eines Bestandteils	≈ 35 %	**Bände 20–22**
An den Nähten	**≈ 45 %**	**dieser Band**
Im Betrieb	≈ 20 %	Kap. 12

Fast die Hälfte der Fehler eines zusammengesetzten Systems entsteht dort, wo niemand zuständig ist — zwischen zwei Bestandteilen, für die je ein anderes Team die Verantwortung trägt. Das ist keine Nachlässigkeit, sondern eine Folge der Aufteilung.

Der Irrtum, der diesen Band nötig macht

Annahme	Zutreffend?	
Ein Aufruf über die Naht ist wie ein Aufruf im Programm	**nein**	**Kap. 2**
Er kommt an	nein	
Er kommt einmal an	**nein**	
Er kommt in der Reihenfolge an	nein	
Er dauert etwa gleich lange	**nein**	Kap. 7

Ein Funktionsaufruf innerhalb eines Programms erfüllt alle fünf Annahmen. Ein Aufruf über eine Naht erfüllt keine einzige — und der Entwurf, der so tut, als wäre es dasselbe, bricht bei der ersten Netzstörung.

Dieser Irrtum ist so alt wie die verteilte Programmierung und so verbreitet wie eh. Er hat einen Namen, eine Literatur und eine Liste von acht Trugschlüssen — Kapitel 2 geht sie durch.

Was dieser Band voraussetzt

Band 20	Der zweite Entwurf — Zerlegung, Prüfen, Aufhören
Band 21	**Was darunter liegt — Speicher, Zahlen, Grenzen**
Band 22	Die Kunst des Weglassens — Nebenläufigkeit, Nähte in Go
Hier	**Was gilt, wenn keine Sprache mehr die Antwort gibt**

Die letzte Zeile ist die Klammer: Zeit, Text, Zahlen, Fehler und Ausfall müssen durch jede Naht hindurch — und keine Sprache beantwortet, in welcher Form. Das entscheidet, wer die Naht entwirft.

Das Vorhaben, vollständig.

Bestandteil	Sprache	Merkmal	Band
Messerfassung	**C**	2 ms hart	21
Übertragung	**Go**	1000 Verb.	22
Speicherung	**SQL**	Historie	12
Auswertung	**Python**	Fachlogik	20
Oberfläche	Web	—	18

Fünf Bestandteile, vier Nähte, und jede Naht ist ein Ort, an dem ein Messwert seine Bedeutung verlieren kann. Der Band folgt einem Kaliumwert von der Messung bis zur Warnung und fragt an jeder Naht, was ihm zustoßen kann.

Was dieses Buch will

Erstens: die Naht zum Entwurfsgegenstand machen.

> *In den meisten Vorhaben entsteht eine Schnittstelle nebenbei — jemand braucht Daten, jemand liefert sie, und das Format ist, was gerade herauskam. Dieser Band behandelt sie als das, was sie ist: die langlebigste Entscheidung des ganzen Systems.*

Zweitens: die Größen benennen, die durch jede Naht müssen.

Zeitpunkt, Zeitzone, Kodierung, Einheit, Genauigkeit, Kennung. Sechs Größen, die in jedem Bestandteil anders dargestellt sind — und deren Umrechnung an der Naht stattfindet oder gar nicht.

Drittens: sagen, wann man nicht teilen sollte.

Jede Naht kostet. Kapitel 14 rechnet nach, was ein zusätzlicher Bestandteil an Aufwand, Fehlern und Verzögerung mit sich bringt — und wann ein einziges Programm die bessere Antwort ist.

Drei Warnungen

Erstens: Aufteilen löst keine Entwurfsprobleme.

> *Ein schlecht geschnittenes Programm wird durch Aufteilen zu einem schlecht geschnittenen System — mit denselben Abhängigkeiten, nun über das Netz. Die Zerlegungsfrage aus Band 17 kommt vor der Verteilungsfrage.*

Zweitens: Die Naht überlebt die Bestandteile.
Ein Datenformat, das zwanzig Bestandteile benutzen, lässt sich nicht mehr ändern. Man kann jeden Bestandteil neu schreiben; das Format bleibt. Deshalb verdient es die Sorgfalt, die man sonst dem Code widmet.
Drittens: Der Betrieb ist Teil des Entwurfs.

Ein System, das man im Betrieb nicht befragen kann, ist nicht fertig entworfen. Kapitel 11 behandelt das als Entwurfsfrage, nicht als nachträgliche Zutat.

Zum Aufbau

- **Teil I** (Kap. 1–3): Das Werk als Ganzes. Programm gegen Werk, **die Naht**, Verträge.
- **Teil II** (Kap. 4–6): Über die Grenze. **Zeit**, Text, Zahlen.
- **Teil III** (Kap. 7–9): Leistung im Großen. Messen, **wo die Zeit hingeht**, Zwischenspeicher.
- **Teil IV** (Kap. 10–12): Der Betrieb. Fehler, **Beobachtbarkeit**, Ausfall.
- **Teil V** (Kap. 13–15): Das Urteil. Der zweite Entwurf, **wann teilen**, der Stand 2026.

Ein Kaliumwert von 5,1

Ein Laborgerät misst Kalium und meldet 5,1. Der Wert erreicht die Station als 51.

Das Gerät	Sendet ganzzahlig in Einheiten der letzten Stelle: 51
Die Naht	**Der Exponent stand in einem Feld, das niemand las**
Der Empfänger	Nahm den Wert als Zahl und zeigte 51 an
Der Bereich	3,5 bis 5,1 — also “stark erhöht”

Beide Programme sind richtig. Das Gerät sendet, was vereinbart war; der Empfänger liest, was er versteht. Der Fehler sitzt in der Naht — in einer Vereinbarung, die einer der beiden Seiten nicht bekannt war.

Was den Fall verhindert hätte.

Maßnahme	Aufwand	Wirkung
Einheit mitsenden	gering	Prüfung möglich
Wert und Exponent als ein Feld	**gering**	**untrennbar**
Plausibilitätsgrenze	gering	51 ist unmöglich
Gemeinsames Schema	mittel	beide prüfen

Die dritte Zeile ist die billigste und wird am häufigsten unterlassen: Ein Kaliumwert von einundfünfzig ist mit dem Leben nicht vereinbar. Eine Grenzprüfung an der Naht hätte den Wert zurückgewiesen, statt ihn anzuzeigen.

Der Fall ist erfunden und in seiner Bauart alltäglich. Er zeigt, worum es in diesem Band geht: Nicht darum, besseren Code zu schreiben, sondern darum, die Vereinbarungen zwischen gutem Code zu entwerfen.

Ein Programm kann man neu schreiben.
Eine Naht muss man aushandeln —
mit allen, die daran hängen.

— der Leitgedanke dieses Bandes.
Deshalb ist die Naht die teuerste Entscheidung
und bekommt die geringste Aufmerksamkeit.

Teil I

Das Werk als Ganzes

Kapitel 1

Programm und Werk

Der Unterschied

	Programm	Werk
Läuft	**einmal, irgendwo**	**dauernd, an vielen Orten**
Fehler	Absturz	**Teilausfall**
Ändern	neu übersetzen	im laufenden Betrieb
Zustand	im Speicher	**verteilt, dauerhaft**
Zuständigkeit	**einer**	**mehrere**

Die zweite Zeile ist der wichtigste Unterschied: Ein Programm läuft oder es läuft nicht. Ein Werk kann zu vierzig Prozent laufen — und diese Möglichkeit ist der Ursprung fast aller Schwierigkeiten, die dieser Band behandelt.

Was der Teilausfall bedeutet.

Lage	Im Programm	Im Werk
Aufruf schlägt fehl	Fehlerwert	**oder Stille**
Empfänger weg	gibt es nicht	Normalfall
Antwort verspätet	gibt es nicht	**nicht von “weg” zu trennen**
Halb ausgeführt	selten	**häufig**

Die dritte Zeile ist die grundlegende: Ein Aufruf, der nicht antwortet, kann verlorengegangen, unterwegs oder ausgeführt und die Antwort verloren sein. Der Sender kann die drei Fälle nicht unterscheiden — und muss trotzdem handeln.

Die Größenordnungen

Vorgang	Dauer	
Funktionsaufruf	≈ 2 ns	
Sprachgrenze	≈ 60 ns	**Band 21, Kap. 14**
Prozessgrenze	≈ 5 µs	Rohr, Steckdose
Rechner im Haus	≈ 0,5 ms	**Faktor 250 000**
Rechenzentrum entfernt	≈ 80 ms	

Zwischen einem Funktionsaufruf und einem Aufruf über das Hausnetz liegt Faktor zweihundertfünfzigtausend. Wer eine Naht dort legt, wo vorher ein Aufruf stand, ändert nicht die Struktur, sondern die Größenordnung — und Größenordnungen ändern die richtige Bauform.

Die Zerlegung kommt vor der Verteilung

Zuerst	Wonach schneidet man? — Parnas' Frage aus Band 17
Dann erst	**Was davon läuft in einem eigenen Prozess?**
Der Fehler	Beides zugleich entscheiden

Ein schlecht geschnittenes Programm wird durch Aufteilen zu einem schlecht geschnittenen Werk — mit denselben Abhängigkeiten, nun über das Netz, mit Latenz und Teilausfall. Man hat nichts gewonnen und fünf Schwierigkeiten hinzugefügt.

Die Probe.

Frage	Antwort	
Ließe sich der Teil als Modul sauber trennen?	**muss ja sein**	**sonst nicht verteilen**
Ändert er sich aus eigenem Grund?	muss ja sein	
Hat er eigene Betriebsan-forderungen?	dann lohnt es	
Nur weil es modern ist?	**nein**	

Die dritte Zeile ist der beste positive Grund: Wenn ein Teil andere Zeitschranken, andere Verfügbarkeit oder anderen Speicherbedarf hat als der Rest, gehört er in einen eigenen Prozess. Beim Messdatenerfasser ist das offensichtlich; bei einer Auswertungsfunktion meist nicht.

Conways Beobachtung

Die Aussage	Ein System bildet die Kommunikationsstruktur seiner Erbauer ab
Die Folge	**Drei Teams erzeugen drei Bestandteile — auch wenn zwei genügten**
Der Umkehrschluss	Wer die Struktur ändern will, ändert zuerst die Zuständigkeiten

Die Beobachtung ist fast sechzig Jahre alt und in jedem größeren Vorhaben nachprüfbar. Sie erklärt, warum Nähte oft dort verlaufen, wo Abteilungs-grenzen liegen — und nicht dort, wo die Sache es verlangt.

Für ein Klinikvorhaben heißt das konkret: Wenn Labor, Station und Verwaltung je eigene Zuständigkeiten haben, entstehen drei Bestandteile mit drei Nähten — unabhängig davon, ob der Befund das nahelegt.

Übungsaufgaben

(a) Bewerten Sie eine vorgeschlagene Aufteilung.

(b) Rechnen Sie die Kosten einer Naht aus.

(c) Bestimmen Sie, was ein Werk fertig macht.

Lösung

(a) Eine Aufteilung bewerten.
Der Vorschlag. Die Auswertung in vier Dienste zerlegen.

Vorschlag	Eigener Grund?	Eigener Betrieb?	Urteil
Befunddienst	ja	ja	**ja**
Verlaufsdienst	**nein**	nein	**nein**
Anlassdienst	nein	nein	nein
Benachrichtigung	ja	**ja**	ja

Zwei von vier tragen. Verlauf und Anlassbestimmung sind reine Rechnung auf denselben Daten und ändern sich aus demselben Grund — sie gehören in denselben Prozess. Ein eigener Dienst dafür erzeugt zwei Nähte für null Gewinn.

Die Rechnung für den Verlaufsdienst.

	Im Prozess	Als Dienst
Aufrufdauer	**48** ms	**51** ms
Bei Ausfall	unmöglich	**Auswertung steht**
Zeilen zusätzlich	0	≈ 340
Bereitstellungen	1	2
Fehlerklassen neu	0	**5**

Drei Millisekunden mehr, dreihundertvierzig Zeilen Netzcode, eine zusätzliche Ausfallmöglichkeit und fünf neue Fehlerklassen — für einen Bestandteil, der mit dem Aufrufer immer gemeinsam geändert wird.

(b) Kosten einer Naht.

Posten	Einmalig	Laufend
Format vereinbaren	≈ 2 PT	
Beide Seiten umsetzen	≈ 5 PT	
Fehlerbehandlung	≈ 4 PT	
Prüfung über die Naht	≈ 3 PT	**je Änderung**
Betriebs-überwachung	≈ 2 PT	Bereitschaft
Versionierung	≈ 2 PT	**dauerhaft**
Summe	**≈ 18 PT**	

Achtzehn Personentage für eine Naht, bevor eine Zeile Fachlogik geschrieben ist. Bei fünf Bestandteilen und vier Nähten sind das zweiundsiebzig Tage — ein Drittel Personenjahr, das im Zeitplan meist nicht auftaucht.

Die vorletzte Zeile ist die dauerhafteste: Eine Naht muss versioniert werden, weil beide Seiten nicht gleichzeitig umgestellt werden können. Kapitel 3 behandelt, wie.

(c) Was ein Werk fertig macht.

Bedingung	Bei einem Programm	Bei einem Werk
Rechnet richtig	**ja**	ja
Läuft schnell genug	ja	ja
Prüfbar	ja	ja
Übersteht Teilausfall	—	**ja**
Im Betrieb befragbar	—	**ja**
Einzeln erneuerbar	—	**ja**

Drei zusätzliche Bedingungen gegenüber Band 20, Kapitel 14 — und alle drei sind prüfbar. Die fünfte wird am häufigsten vergessen: Ein Werk, das man im Betrieb nicht befragen kann, ist nicht fertig, gleichgültig wie gut es rechnet.

Die sechste ist die anspruchsvollste: Jeder Bestandteil muss einzeln ausgetauscht werden können, während die anderen laufen. Wer das nicht kann, muss für jede Änderung das ganze Werk anhalten — in einer Klinik ein Vorgang mit Terminplanung.

Der Zeitplan lügt bei Werken

Posten	Geschätzt	Tatsächlich
Fachlogik	60 %	**35** %
Nähte	15 %	**30** %
Betrieb, Überwachung	10 %	**20** %
Fehlerfälle	15 %	15 %

Die zweite und dritte Zeile werden regelmäßig halbiert geschätzt, weil sie beim Entwurf unsichtbar sind: Man plant, was das System tun soll, und nicht, wie die Teile miteinander reden und wie man ihnen dabei zusieht.

Brooks' Beobachtung, dass Vorhaben nicht durch mehr Leute schneller werden, hat hier eine Entsprechung: Sie werden auch nicht dadurch schneller, dass man sie in mehr Bestandteile zerlegt. Jede Naht ist Kommunikationsaufwand, in Code gegossen.

Parnas[19] stellt die Zerlegungsfrage, die vor jeder Verteilung steht; Ford und Richards[9] ordnen die Entwurfsentscheidungen im Großen; Brooks[5] beschreibt, warum Aufteilung Kommunikationsaufwand erzeugt.

Kapitel 2

Die Naht

Acht Trugschlüsse

Angenommen wird	Tatsächlich	
Das Netz ist zuverlässig	**nein**	**Kap. 10**
Latenz ist null	≈ 0,5–80 ms	
Bandbreite ist unbegrenzt	nein	
Das Netz ist sicher	**nein**	**Band 19**
Die Struktur ändert sich nicht	nein	
Es gibt einen Zuständigen	**nein**	**Kap. 1**
Übertragungskosten sind null	nein	
Das Netz ist einheitlich	nein	

Die Liste stammt aus den frühen Neunzigern und ist unverändert gültig. Ihr Wert liegt nicht in der Neuigkeit, sondern darin, dass jeder Punkt eine Entwurfsentscheidung verlangt — und dass ein Entwurf, der einen davon übersieht, an genau dieser Stelle bricht.

Was daraus folgt.

Trugschluss	Verlangt	
Zuverlässig	**Wiederholung**	und Idempotenz
Latenz null	**grobe Aufrufe**	keine Schleifen
Zuständiger	**Zeitschranken**	überall
Struktur fest	Namens-auflösung	

Die erste Zeile enthält den Kern des Kapitels: Wiederholung ist die Antwort auf Unzuverlässigkeit — und sie ist nur zulässig, wenn eine zweimal ausgeführte Anfrage dasselbe bewirkt wie eine einmal ausgeführte.

Grobe statt feiner Aufrufe

Bauform	Aufrufe	Dauer
Je Befund einer	**500**	**260 ms**
In Blöcken zu 50	10	9 ms
Ein Aufruf	**1**	**3 ms**

Dasselbe n+1-Problem wie in Band 20, Kapitel 12 — hier mit Faktor achtzig statt vierzig, weil die Naht teurer ist als eine Datenbankabfrage. Die Regel lautet: An einer Naht überträgt man Mengen, nicht Einzelstücke.

Die dritte Zeile hat eine Grenze: Ein einziger Aufruf mit hunderttausend Befunden scheitert an Speicher und Zeitschranken. Zwischen den beiden Fehlern liegt die richtige Blockgröße — und die findet man durch Messen, nicht durch Nachdenken.

Idempotenz

Die Eigenschaft	Zweimal ausgeführt wirkt wie einmal ausgeführt
Warum nötig	**Der Sender weiß nach einem Zeitablauf nicht, ob es ankam**
Die Alternative	Es gibt keine — ohne sie ist Wiederholung unzulässig

Das ist der zentrale Begriff dieses Kapitels. Ohne Idempotenz steht der Sender nach einem Zeitablauf vor einer unlösbaren Wahl: Wiederholen kann einen Wert verdoppeln, nicht wiederholen kann ihn verlieren — und beides ist in der Klinik ein Schaden.

Wie man sie herstellt.

Mittel	Aufwand	
Kennung vom Sender	gering	**das übliche Mittel**
Zustand statt Änderung	gering	**setzen statt erhöhen**
Bedingtes Schreiben	mittel	“nur wenn Fassung *n*”
Empfänger merkt sich Kennungen	Speicher	mit Verfallszeit

Die zweite Zeile ist die eleganteste: “Setze den Kaliumwert für Probe 4711 auf 5,1” ist idempotent; “füge einen Kaliumwert hinzu” ist es nicht. Die Umformulierung kostet nichts und beseitigt die Fehlerklasse.

Die Kennung muss vom Sender kommen

	Empfänger vergibt	Sender vergibt
Bei Wiederholung	**zwei Einträge**	**einer**
Erkennbar	nein	ja
Sender kann nachfragen	nein	**ja**

Wenn der Empfänger die Kennung vergibt, kann er eine Wiederholung nicht von einem neuen Wert unterscheiden — der Sender hat nichts, worauf er sich beziehen könnte. Die Kennung gehört deshalb zur Nachricht, nicht zur Antwort.

Beim Messdatenerfasser ist die natürliche Kennung ohnehin vorhanden: Gerät, Probe, Messgröße und Zeitpunkt bestimmen einen Wert eindeutig. Man muss sie nur mitschicken.

Übungsaufgaben

(a) Entwerfen Sie die Naht zwischen Erfassung und Übertragung.

(b) Bestimmen Sie die Blockgröße.

(c) Untersuchen Sie, wann eine Warteschlange dazwischen gehört.

Lösung

(a) Die Naht entwerfen.

Entscheidung	Gewählt	Warum
Richtung	**Erfassung sendet**	**kennt den Takt**
Form	Blöcke	Kap. oben
Kennung	**Gerät, Probe, Zeit**	**idempotent**
Bestätigung	je Block	Wiederholung möglich
Bei Ausfall	**Erfassung puffert**	Band 21, Kap. 13
Format	**fest, binär**	64 **KB Speicher**

Die letzte Zeile folgt aus der Umgebung: Ein Messdatenerfasser mit vierundsechzig Kilobyte kann kein JSON erzeugen. Das Format der Naht wird von der schwächsten Seite bestimmt — eine Regel, die man beim Entwurf oft übergeht.

Der Vergleich der Formate.

Format	Byte/Wert	Erzeugen	Lesbar
Fest, binär	**16**	**0,2 µs**	nein
Selbstbeschreibend	34	1,1 µs	mit Werkzeug
JSON	**142**	**4,8 µs**	**ja**
XML	310	9,2 µs	ja

Faktor neun im Umfang und vierundzwanzig beim Erzeugen zwischen der ersten und der dritten Zeile. An der ersten Naht entscheidet das; an der Naht zur Auswertung, wo Menschen mitlesen, entscheidet die letzte Spalte.

Daraus folgt eine Erkenntnis, die viele überrascht: Ein Werk hat nicht ein Format, sondern eines je Naht. Die Umwandlung geschieht an der Übertragungsschicht — das ist eine ihrer Aufgaben.

(b) Die Blockgröße.

Block	Durchsatz	Verzögerung	Verlust bei Ausfall
1	3,8 k/s	**0,5** ms	1 Wert
50	185 k/s	**28** ms	50
500	**495 k/s**	**95** ms	500
5000	510 k/s	**940** ms	5000

Die dritte Zeile ist das Optimum: Der Durchsatz ist fast erreicht, die Verzögerung liegt unter der weichen Schranke von einer Sekunde, und der Verlust bei einem Ausfall bleibt überschaubar. Die vierte Zeile bringt drei Prozent Durchsatz und verzehnfacht beides.

Die letzte Spalte wird beim Entwurf oft vergessen und ist klinisch die wichtigste: Ein Block ist die Einheit, die bei einem Absturz verlorengeht — wenn die Erfassung nicht dauerhaft puffert.

(c) Wann eine Warteschlange dazwischen gehört.

Lage	Warteschlange?	
Empfänger langsamer	**ja**	**Lastspitzen**
Empfänger fällt aus	**ja**	Werte bleiben
Mehrere Empfänger	ja	
Antwort wird gebraucht	**nein**	direkt aufrufen
Zwei Bestandteile, fest	nein	**Aufwand ohne Ertrag**

Die zweite Zeile ist der stärkste Grund: Eine Warteschlange entkoppelt die Verfügbarkeit. Der Sender kann weiterarbeiten, wenn der Empfänger steht — und in einem Klinikbetrieb, in dem Wartungsfenster vorkommen, ist das keine Kleinigkeit.

	Direkt	Warteschlange
Bestandteile	2	**3**
Verzögerung	**3** ms	12 ms
Bei Empfängerausfall	Verlust	**Puffer**
Betriebsaufwand	gering	eigenes System
Reihenfolge	sicher	je nach System

Die letzte Zeile ist eine unangenehme Überraschung: Viele Warteschlangensysteme sichern die Reihenfolge nur innerhalb eines Abschnitts zu. Wer Verläufe überträgt, muss das wissen — oder die Reihenfolge selbst aus den Zeitpunkten wiederherstellen.

Die Naht ist keine Funktion

Versuchung	Folge	
Aufrufe wie im Code	$n{+}1$ **über das Netz**	
Ausnahmen durchreichen	geht nicht	**Band 21, Kap. 14**
Objekte übertragen	**Kopplung**	**beide Seiten hängen**
Zeiger übertragen	sinnlos	

Die dritte Zeile ist die folgenreichste: Wer die inneren Datenstrukturen eines Bestandteils überträgt, koppelt beide Seiten an eine Umsetzung. Jede Änderung der einen erzwingt eine Änderung der anderen — genau das, was die Naht verhindern sollte.

Die Naht überträgt Fachbegriffe, keine Objekte: einen Befund mit Wert, Einheit, Zeitpunkt und Kennung — nicht die Klasse, mit der ein Bestandteil ihn zufällig darstellt.

Waldo und Mitautoren[25] beschrieben 1994, warum ein entfernter Aufruf kein Aufruf ist; Hohpe und Woolf[12] sammeln die Bauformen der Verbindung; Parnas[19] liefert den Maßstab, wonach geschnitten wird.

Kapitel 3

Verträge

Was ein Nahtvertrag festlegt

Punkt	Beispiel	
Felder und Typen	`wert: int64`	**das Offensichtliche**
Einheit	`mmol/l`	**oft fehlend**
Genauigkeit	`exp: -2`	oft fehlend
Pflicht oder optional	—	
Wertebereich	0 bis 20	**Plausibilität**
Bedeutung des Fehlens	**nicht gemessen?**	**nie festgelegt**
Verhalten bei Wiederholung	idempotent	

Sieben Punkte, von denen die meisten Schemasprachen nur den ersten und vierten ausdrücken können. Die übrigen stehen im besten Fall im Kommentar — und im Normalfall nirgends.

Die sechste Zeile im Einzelnen.

Feld fehlt	Kann bedeuten: nicht gemessen, nicht auswertbar, nicht übermittelt, nicht erlaubt
Der Unterschied	**Klinisch erheblich — Band 20, Kap. 2**
Die Abhilfe	Kein Weglassen, sondern ein ausdrücklicher Wert

Ein fehlender Kaliumwert und ein nicht auswertbarer Kaliumwert sind verschiedene Sachverhalte. Wer beide durch "Feld nicht vorhanden" darstellt, hat Information vernichtet, bevor sie die Naht erreicht.

Was ein Schema leistet und was nicht

Prüft	Schema	
Feld vorhanden	**ja**	
Typ richtig	ja	
Wertebereich	teils	
Einheit passt zur Größe	**nein**	**Fachlogik**
Zeitpunkt plausibel	nein	
Kennung eindeutig	nein	

Ein Schema ist die untere Hälfte des Vertrags — notwendig und bei weitem nicht hinreichend. Die drei unteren Zeilen verlangen fachliche Prüfungen an der Naht, und die schreibt niemand, wenn das Schema grün meldet.

Änderungen ohne Bruch

Änderung	Bricht?	
Optionales Feld hinzu	**nein**	**der sichere Weg**
Pflichtfeld hinzu	**ja**	alte Sender
Feld entfernen	ja	alte Empfänger
Typ ändern	**ja**	
Feld umbenennen	ja	**= entfernen + hinzu**
Bedeutung ändern	**ja, unbemerkt**	**das Schlimmste**

Die letzte Zeile ist die gefährlichste, weil kein Werkzeug sie erkennt: Wenn `zeitpunkt` von der Abnahme auf die Validierung umgedeutet wird, bleibt das Schema gültig und alle Verläufe sind falsch. Kapitel 4 behandelt genau diesen Fall.

Die Regel für beide Seiten.

Als Sender	Sende genau, was vereinbart ist — streng
Als Empfänger	**Nimm entgegen, was du verstehst; ignoriere Unbekanntes**
Die Grenze	**Nicht: rate, was gemeint war**

Postels Regel, streng gelesen. Sie erlaubt, ein neues optionales Feld einzuführen, ohne alle Empfänger gleichzeitig umzustellen. Die dritte Zeile ist der heute übliche Vorbehalt: Großzügigkeit beim Deuten hat mehr Sicherheitslücken erzeugt als Nachsicht Nutzen brachte.

Die Reihenfolge beim Umstellen

1	**Empfänger versteht altes und neues Format**
2	Empfänger ausrollen, warten, prüfen
3	**Sender auf neues Format umstellen**
4	Warten, bis kein altes Format mehr auftritt
5	**Erst dann altes Format entfernen**

Fünf Schritte über Wochen, für eine Änderung, die im Programm eine Minute dauert. Das ist der Preis der Naht — und wer die Schritte zwei und vier überspringt, verliert Daten in dem Zeitfenster, in dem beide Seiten nicht zusammenpassen.

Schritt vier verlangt eine Messung: Wie viele Nachrichten kommen noch im alten Format? Ohne diese Zahl ist Schritt fünf ein Ratespiel — Kapitel 11 liefert sie.

Übungsaufgaben

(a) Schreiben Sie den Vertrag für die Befundnaht.

(b) Bewerten Sie ein fachliches Austauschformat.

(c) Bestimmen Sie, wer die Prüfung leistet.

Lösung

(a) Der Vertrag für die Befundnaht.

```
Befund
kennung     Pflicht  Gerät+Probe+Code+Abnahme
groesse     Pflicht  LOINC-Code
art         Pflicht  zahl|grenze|qualitativ|nichtauswertbar
betrag      bei art=zahl  int64
exponent    bei art=zahl  int8, Zehnerpotenz
einheit     bei art=zahl  UCUM
abnahme     Pflicht  RFC 3339 mit Zonenversatz
eingang     Pflicht  RFC 3339 mit Zonenversatz
validierung optional  RFC 3339, fehlt = nicht validiert
ersetzt     optional  Kennung des ersetzten Befunds
grund       bei art=nichtauswertbar  Text
```

Entscheidung	Gewählt	Warum
Wert	**Betrag + Exponent**	**Band 21, Kap. 8**
Einheit	**eigenes Feld**	prüfbar
Zeitpunkte	**drei getrennt**	Band 20, Kap. 2
Fehlen	**ausdrücklich geregelt**	
Ersetzung	**Verweis**	Historie bleibt

Die dritte Zeile beseitigt den Fall aus dem Vorwort: Wer alle drei Zeitpunkte überträgt, zwingt den Empfänger zur Entscheidung, welchen er meint — statt ihn raten zu lassen. Ein Feld mehr, eine Fehlerklasse weniger.

Die vorletzte Zeile ist der Satz, der am häufigsten fehlt: "Fehlt `validierung`, so ist der Befund nicht validiert" — eine Zeile, die entscheidet, ob ein vorläufiger Wert als endgültig behandelt wird.

(b) Ein fachliches Austauschformat.

	Eigenes Format	Fachstandard
Aufwand anfangs	**gering**	hoch
Passt genau	ja	**überladen**
Fremdsysteme	**Übersetzer je System**	**direkt**
Codelisten vorhanden	nein	**LOINC, UCUM**
Regulatorisch	begründungs-pflichtig	erwartet
Werkzeuge	selbst	vorhanden

Die vierte Zeile ist der unterschätzte Gewinn: Ein Fachstandard bringt die Codelisten mit — welche Messgröße welchen Code hat, welche Einheiten zulässig sind. Diese Listen selbst zu pflegen ist teurer als das ganze Format.

Die zweite Zeile ist der Preis: Ein Fachstandard, der viele Häuser bedienen muss, hat für jeden einzelnen zu viele Felder und zu viel Freiheit. Die übliche Antwort ist eine Einschränkung — ein Profil, das festlegt, welcher Teil verwendet wird.

Empfehlung	**Wo**	
Fachstandard	**nach außen**	andere Häuser
Eigenes Format	**innen**	Erfassung, Übertragung
Übersetzung	**an einer Stelle**	**prüfbar**

(c) Wer die Prüfung leistet.

Prüfung	**Sender**	**Empfänger**
Schema	ja	**ja**
Wertebereich	ja	**ja**
Einheit passt	ja	ja
Kennung eindeutig	kann nicht	**ja**
Zeitpunkt plausibel	ja	ja

Beide Seiten prüfen — das ist keine Verschwendung, sondern die Regel. Der Sender prüft, um keinen Unsinn zu senden; der Empfänger prüft, weil er dem Sender nicht trauen darf. Band 19 nennt das Vertrauensgrenze.

Nur Sender prüft	**Folge**	
Fehlerhafter Sender	**Unsinn gespeichert**	
Neuer Sender	prüft anders	
Angriff	**ungeprüft**	**Band 19, Kap. 4**

Die erste Zeile ist der Alltagsfall: Ein Gerät bekommt eine neue Betriebsfassung und sendet plötzlich Werte in anderer Einheit. Der Empfänger, der nicht prüft, speichert sie — und die Warnungen kommen für Wochen falsch.

Was bei Zurückweisung geschieht

Vorgehen	Tauglich?	
Stillschweigend verwerfen	**nie**	**Datenverlust**
Ganzen Block ablehnen	teils	ein Fehler blockiert 500
Fehlerhaften Wert markieren	**ja**	**Rest wird gespeichert**
In Sonderablage	ja	**mit Meldung**

Die dritte Zeile ist die richtige Antwort und verlangt, dass der Vertrag den Fall vorsieht: Der Empfänger nimmt den Block an, speichert die gültigen Werte und meldet die ungültigen einzeln zurück. Ohne diese Vorsehung bleibt nur die zweite Zeile.

Die vierte Zeile ist die Betriebsseite: Zurückgewiesene Werte verschwinden nicht, sondern landen in einer Ablage, die jemand ansieht. Eine Ablage, die niemand ansieht, ist dasselbe wie die erste Zeile.

Die FHIR-Spezifikation[27] zeigt einen ausgearbeiteten Fachvertrag samt Codelisten; Evans[7] begründet, warum Fachbegriffe und nicht Objekte über die Naht gehen; Postels Regel[20] legt fest, wie streng jede Seite sein soll.

Teil II

Über die Grenze

Kapitel 4

Zeit

Sechs Zeitangaben, die verwechselt werden

Angabe	Bedeutet	
Abnahme	**wann entnommen**	**fachlich richtig**
Eingang	wann im Labor	
Messung	wann analysiert	
Validierung	**wann freigegeben**	klinisch gültig
Übertragung	wann gesendet	technisch
Speicherung	wann geschrieben	**oft die einzige**

Die letzte Zeile ist der Normalfall in gewachsenen Systemen: Gespeichert wird der Zeitpunkt des Schreibens, weil die Datenbank ihn ohnehin setzt. Ein Verlauf, der darauf beruht, zeigt die Reihenfolge der Übertragung — nicht die der Abnahme.

Wo der Unterschied wirkt.

Lage	Abstand	Folge
Regelbetrieb	≈ 45 min	unauffällig
Nachtdienst	≈ 3 h	spürbar
Nacherfassung	**2 Tage**	**Verlauf falsch**
Fremdhaus	Wochen	**Reihenfolge verdreht**

Die dritte Zeile ist der Fall, an dem es auffällt: Ein zwei Tage alter Befund wird nacherfasst und erscheint im Verlauf als der neueste. Die Steigungsprüfung schlägt an, die Warnung ist falsch — und der Fehler liegt in der Wahl des Zeitpunkts, nicht in der Rechnung.

Die zwei Zeitachsen

	Fachliche Zeit	Systemzeit
Sagt	**wann es geschah**	**wann wir es wussten**
Kann sich ändern	ja, Korrektur	nie
Für Verläufe	**ja**	nein
Für Nachvollzug	nein	**ja**

Beide werden gebraucht, und für verschiedene Fragen. "Wie war der Kaliumverlauf?" braucht die fachliche Zeit. "Was wusste der Arzt am Dienstag um 14 Uhr?" braucht die Systemzeit — und diese Frage stellt sich bei jeder Nachprüfung eines Behandlungsverlaufs.

Ein System, das nur eine der beiden führt, kann eine der beiden Fragen nicht beantworten. Der Aufwand für beide ist ein zusätzliches Feld je Datensatz — und die Antwort auf eine Frage, die in der Klinik regelmäßig vor Gericht gestellt wird.

Zeitzonen

Darstellung	Eindeutig?	
`2026-03-29 02:30`	**nein**	**existiert nicht**
`2026-10-25 02:30`	**nein**	zweimal vorhanden
`...02:30+01:00`	**ja**	**Versatz**
`...02:30Z`	ja	
Unix-Zeit	ja	Ortszeit verloren
`...+01:00` und `Europe/Berlin`	**ja**	**vollständig**

Die ersten beiden Zeilen sind kein Sonderfall, sondern kommen zweimal im Jahr vor. In der Nacht der Rückstellung gibt es zwei Uhrzeiten `02:30` — und ein Verlauf, der nur Ortszeit führt, kann sie nicht ordnen.

Warum der Versatz nicht genügt.

Frage	Versatz	Zonenname
Wann war es?	**ja**	ja
Ortszeit anzeigen	ja	ja
Termin in 6 Monaten	**nein**	**ja**
Nach Umstellungsregeländerung	nein	ja

Für Vergangenes genügt der Versatz — er sagt, was war. Für Zukünftiges nicht: Ein Termin am 15. Juli um 9 Uhr Ortszeit hat einen Versatz, den man erst kennt, wenn feststeht, ob die Umstellung dann noch gilt. Termine speichert man als Ortszeit mit Zonennamen.

Die Uhren gehen verschieden

Abweichung	Größe	
Mit Zeitabgleich	< 10 ms	gut gepflegt
Ohne Abgleich	≈ 1 s/Tag	
Eingebettetes Gerät	**5–30 s/Tag**	**Messerfasser**
Nach Stromausfall	**beliebig**	**1970**

Die dritte Zeile ist der praktische Fall: Ein Messdatenerfasser ohne Netzanschluss und ohne Pufferbatterie hat keine verlässliche Uhr. Nach vier Wochen liegt er zwei Minuten daneben — genug, um zwei Messungen in der falschen Reihenfolge zu ordnen.

Die vierte Zeile ist die schlimmere: Nach einem Stromausfall startet manches Gerät mit dem 1. Januar 1970. Werte mit diesem Zeitpunkt gehören zurückgewiesen — eine Plausibilitätsprüfung, die drei Zeilen kostet.

Übungsaufgaben

(a) Ordnen Sie Werte ohne verlässliche Uhr.

(b) Bestimmen Sie die Darstellung an jeder Naht.

(c) Untersuchen Sie den Umgang mit Korrekturen.

Lösung

(a) Ordnen ohne verlässliche Uhr.
Das Problem. Der Erfasser weiß, in welcher Reihenfolge er misst, aber nicht, wann.

Mittel	Leistet	
Fortlaufende Nummer	**Reihenfolge**	**zuverlässig**
Laufzeit seit Start	Abstände	genau
Startzeitpunkt	**Verankerung**	**vom Empfänger**
Gerätekennung	Zuordnung	

Der Kunstgriff: Das Gerät sendet Nummer und Laufzeit seit dem Start; die Übertragungsschicht verankert den Start an ihrer eigenen, abgeglichenen Uhr. Damit sind Reihenfolge und Abstände vom Gerät und der absolute Zeitpunkt vom Empfänger — jeder liefert, was er sicher weiß.

Verfahren	Reihenfolge	Zeitpunkt
Geräteuhr	**unsicher**	±2 min
Empfängeruhr	Netzverzug	±1 s
Nummer und Verankerung	**sicher**	**±50 ms**

Faktor zweitausend bei der Genauigkeit gegenüber der Geräteuhr, und die Reihenfolge wird von unsicher zu sicher. Der Aufwand sind zwei zusätzliche Felder in der Nachricht.

(b) Die Darstellung je Naht.

Naht	Darstellung	Warum
Erfassung → Übertragung	**Nummer + Laufzeit**	**keine Uhr**
Übertragung → Speicher	**RFC 3339, UTC**	eindeutig
Speicher intern	**Zeitstempel mit Zone**	sortierbar
Speicher → Auswertung	RFC 3339	
Auswertung → Anzeige	**Ortszeit + Zone**	**für Menschen**

Fünf Nähte, vier Darstellungen — und das ist kein Mangel, sondern richtig. Jede Naht hat andere Anforderungen: Das Gerät hat keine Uhr, die Datenbank braucht Sortierbarkeit, der Mensch braucht Ortszeit.

Die Regel lautet: intern immer UTC mit Zonenangabe, an den Rändern umwandeln. Wer Ortszeit durch das System reicht, hat an jeder Stelle die Frage zu beantworten, welche Zone gemeint war.

(c) Korrekturen.

Vorgehen	Nachvollziehbar?	
Wert überschreiben	**nein**	**regulatorisch unzulässig**
Alten löschen, neuen anlegen	nein	
Neuen mit Verweis anlegen	**ja**	**Band 20, Kap. 2**
Beide behalten, Gültigkeit setzen	ja	

Die dritte Zeile ist dieselbe Bauform wie in Band 20 und Band 22: Es wird nichts geändert, nur hinzugefügt. Der korrigierte Befund verweist auf den ersetzten, und die Systemzeit sagt, ab wann die Korrektur bekannt war.

Was der Verlauf dann zeigt.

Frage	Antwort aus	
Aktueller Verlauf	**fachliche Zeit, gültige**	
Verlauf am Dienstag	**beide Zeiten**	**rekonstruierbar**
Wurde korrigiert?	Verweiskette	

Die zweite Zeile ist die Frage, die bei jeder Nachprüfung gestellt wird: Welche Werte lagen dem Arzt am Dienstag vor? Sie ist nur beantwortbar, wenn beide Zeitachsen geführt werden — und sie ist nicht nachträglich herstellbar.

Die Sekunde, die es nicht gibt

Fall	Wirkung	
Schaltsekunde	`23:59:60`	**viele Systeme stürzen ab**
Uhr springt zurück	**negative Dauer**	Zeitschranken brechen
Abgleich springt	Sprung	

Die zweite Zeile ist der praktische Fall: Wer eine Dauer als Differenz zweier Uhrzeiten berechnet, bekommt bei einem Rücksprung eine negative Zahl — und in einem vorzeichenlosen Typ eine sehr große. Der Fehler aus Band 21, Kapitel 8, hier über die Uhr.

Die Abhilfe ist eine getrennte Uhr für Dauern: eine, die nur vorwärts läuft und nicht abgeglichen wird. Fast jede Sprache bietet sie, und fast niemand benutzt sie — weil die gewöhnliche Uhr im Versuch dasselbe liefert.

Die Zeitdarstellung nach RFC 3339[14] legt das Format fest; Tanenbaum und van Steen[22] behandeln Uhren und Ordnung in verteilten Systemen; die FHIR-Spezifikation[27] zeigt, welche Zeitpunkte ein Befund fachlich trägt.

Kapitel 5

Text

Drei Bedeutungen von “Länge”

Gezählt wird	“Müller”	Wofür
Bytes (UTF-8)	**7**	**Speicher, Puffer**
Zeichen	6	Feldgrenzen
Sichtbare Einheiten	6	**Anzeige**
Darstellungsbreite	6	Tabellen

Bei einem deutschen Namen fallen drei der vier Zahlen zusammen und nur die Bytezahl weicht ab. Bei einem Namen mit kombinierenden Zeichen oder außerhalb der lateinischen Schrift weichen alle vier voneinander ab — und ein Feld mit “höchstens 40 Zeichen” ist mehrdeutig.

Was an der Naht schiefgeht.

Fehler	Ursache	Folge
Abschneiden mitten im Zeichen	**Bytegrenze**	**ungültiger Text**
Feld zu kurz	Zeichen gegen Bytes	Name gekürzt
Kodierung geraten	**keine Angabe**	**Umlaute zerstört**
Vergleich schlägt fehl	**Normalform**	**derselbe Name, ungleich**
Sortierung falsch	Bytefolge	Ö nach Z

Die vierte Zeile ist die tückischste, weil sie unsichtbar ist: “Müller” kann als ein Zeichen ü oder als u plus kombinierendes Trema kodiert sein. Beide sehen gleich aus, beide sind gültig, und der Bytevergleich sagt “ungleich”.

Normalisieren an der Naht

Form	Tut	
NFC	**zusammensetzen**	**die übliche Wahl**
NFD	zerlegen	macOS-Dateinamen
NFKC	Verträglichkeit	ändert Bedeutung
Keine	beides möglich	Vergleiche brechen

Die Regel: Beim Eintritt in das Werk wird in NFC normalisiert, danach nie wieder. Wer an mehreren Stellen normalisiert, hat dieselbe Wirkung; wer es nirgends tut, bekommt Namen, die sich nicht wiederfinden lassen.

Die dritte Zeile ist eine Warnung: Die Verträglichkeitsformen wandeln auch "fi" in "fi" und römische Ziffern in Buchstaben. Für die Suche brauchbar, für die Speicherung eines Namens nicht.

Der Namensvergleich

Zweck	Verfahren	
Gleichheit prüfen	**NFC, dann Bytes**	exakt
Suchen	**Faltung, Akzente weg**	"Muller" findet "Müller"
Sortieren	**Sortierregeln**	**sprachabhängig**
Patienten zusammenführen	**nie über Namen**	**nur über Kennung**

Die dritte Zeile überrascht: Die richtige Sortierung von "ä" hängt von der Sprache ab — im Deutschen wie "a" oder wie "ae", im Schwedischen nach "z". Es gibt keine sprachneutrale richtige Sortierung, nur eine gewählte.

Die letzte Zeile ist die klinisch wichtigste Regel des Kapitels: Zwei Patienten mit gleichem Namen und Geburtsdatum sind nicht derselbe Patient. Eine Zusammenführung über Namensähnlichkeit ist ein Behandlungsrisiko, keine Bequemlichkeit.

Was in einen Namen alles passt

Annahme	Zutreffend?	
Hat Vor- und Nachnamen	**nein**	**einteilige Namen**
Ändert sich nicht	nein	Heirat, Geschlecht
Passt in 40 Zeichen	**nein**	
Nur lateinische Buchstaben	nein	
Groß- und Kleinschreibung eindeutig	**nein**	**türkisches i**

Die letzte Zeile ist ein Lehrstück: Im Türkischen wird "i" zu "İ" und "I" zu "ı". Eine Umwandlung in Großbuchstaben nach Systemsprache liefert je nach Einstellung verschiedene Ergebnisse — und ein Vergleich schlägt fehl, je nachdem, wo der Rechner steht.

Deshalb wird für technische Vergleiche nie die Systemsprache verwendet, sondern eine festgelegte — oder man vergleicht ohne Umwandlung.

Übungsaufgaben

(a) Legen Sie die Textbehandlung je Naht fest.

(b) Rechnen Sie die Kosten der Umwandlung aus.

(c) Bestimmen Sie, wie Freitext behandelt wird.

Lösung

(a) Textbehandlung je Naht.

Naht	Kodierung	Normalform	Grenze
Erfassung	**ASCII**	—	**Bytes**
Übertragung	**UTF-8**	**NFC**	Bytes
Speicher	UTF-8	NFC	Zeichen
Auswertung	UTF-8	schon NFC	—
Anzeige	UTF-8	—	Breite

Die erste Zeile ist eine bewusste Einschränkung: Der Messdatenerfasser überträgt nur Kennungen und Codes, keine Namen — damit braucht er keine Kodierungsbehandlung. Wer ihm Freitext zumutet, handelt sich das ganze Kapitel ein.

Die zweite Zeile enthält die einzige Normalisierung im ganzen Werk. Sie geschieht an der Eintrittsstelle, und danach kann sich jeder darauf verlassen — die Voraussetzung dafür, dass die vierte Zeile leer bleibt.

(b) Kosten der Umwandlung.

Vorgang	**je Name**	**bei** 10^6	
Prüfen auf gültiges UTF-8	**40** ns	40 ms	
Prüfen auf NFC	**90** ns	90 ms	meist schon
Normalisieren	**620** ns	**620** ms	nur wenn nötig
Faltung für Suche	310 ns	310 ms	

Der Kunstgriff steht in der zweiten Zeile: Die Prüfung, ob ein Text bereits in NFC vorliegt, ist siebenmal billiger als die Normalisierung — und sie fällt bei fast allen Eingaben positiv aus. Man normalisiert nur, wenn die Prüfung fehlschlägt.

Damit kostet die Behandlung im Regelfall hundertdreißig statt sechshundertzwanzig Nanosekunden — und bei einer Million Namen eine Zehntelsekunde statt einer halben.

(c) Freitext.

Frage	**Regel**	
Länge begrenzen	**ja**	**in Bytes und Zeichen**
Steuerzeichen	**verwerfen**	außer Zeilenumbruch
Auszeichnung erlaubt	nein	**Band 19**
Beim Abschneiden	**an Zeichengrenze**	und markieren
Richtungswechsel	**gefährlich**	**Anzeige täuschbar**

Die letzte Zeile ist eine Sicherheitsfrage aus Band 19: Es gibt unsichtbare Zeichen, die die Leserichtung umkehren — damit lässt sich ein Text so anzeigen, dass er etwas anderes bedeutet, als er enthält. In einem Befundkommentar ist das ein ernsthaftes Risiko.

Die Abhilfe ist eine Positivliste: Erlaubt sind Buchstaben, Ziffern, übliche Satzzeichen und Zeilenumbrüche. Alles andere wird an der Naht entfernt — und dass etwas entfernt wurde, wird gemeldet.

Codes statt Text

	Freitext	Code
Auswertbar	**kaum**	**ja**
Sprachabhängig	ja	nein
Vergleichbar	nein	ja
Kodierungs-fragen	**alle**	**keine**
Für den Menschen	ja	Übersetzung nötig

Die vierte Zeile ist der stille Gewinn: Ein Code aus einer festgelegten Liste hat keine Kodierungs-, keine Normalisierungs- und keine Sortierfrage. Das ganze Kapitel entfällt für alles, was sich kodieren lässt.

Deshalb lautet die Entwurfsregel: An der Naht so viel wie möglich kodiert, so wenig wie nötig als Text. Freitext bleibt für das, was man wirklich nicht kodieren kann — und das ist weniger, als man zunächst annimmt.

Der Unicode-Standard[23] legt Normalformen und Vergleichsregeln fest; Goldberg[10] zeigt am Gleitkomma, wie Fragen der Darstellung zu Rechenfehlern werden; die FHIR-Spezifikation[27] liefert die Codelisten, die Freitext ersparen.

Kapitel 6

Zahlen

Was eine Zahl an der Naht braucht

Angabe	Ohne sie	
Betrag	—	
Einheit	**mg/dl oder mmol/l?**	Faktor ≈ 18
Genauigkeit	Stellen erfunden	**Kap. 3**
Bezugsgröße	"pro was"	
Nachweisgrenze	**"< 0,1" als** 0,1	**Band 20, Kap. 2**
Messunsicherheit	selten übertragen	

Die zweite Zeile ist der klassische Fall: Glukose wird in Deutschland in mg/dl und in vielen Ländern in mmol/l angegeben — der Umrechnungsfaktor ist etwa achtzehn. Ein Wert ohne Einheit ist in der Klinik keine Information, sondern eine Gefahr.

Die Einheit gehört zum Wert.

Bauform	Trennbar?	
Zwei Felder	**ja**	eines geht verloren
Einheit implizit	**ja**	**steht nirgends**
Ein Verbund	**nein**	**Band 21, Kap. 5**
Eigener Typ je Größe	nein	Übersetzer prüft

Die dritte und vierte Zeile machen den Fehler unmöglich statt unwahrscheinlich — derselbe Leitgedanke wie in Band 19 und 21. Ein Typ `Konzentration`, der nur mit Einheit erzeugt werden kann, lässt sich nicht versehentlich als bloße Zahl behandeln.

Der Fall aus dem Vorwort, aufgelöst

Gesendet	`{betrag: 51, exponent: -1, einheit: "mmol/L"}`
Bedeutet	5,1 mmol/l
Prüfung 1	**Einheit passt zur Messgröße?** — ja
Prüfung 2	**Wert im Bereich** 0 **bis** 20**?** — ja

Hätte der Empfänger den Exponenten ignoriert, wäre der Wert 51 *— und Prüfung 2 hätte ihn zurückgewiesen. Zwei Prüfungen, von denen jede einzeln den Fall verhindert hätte, und keine davon war vorhanden.*

Das ist die Lehre des Kapitels: Plausibilitätsgrenzen an der Naht kosten wenige Zeilen und fangen genau die Fehler ab, die aus Missverständnissen über die Darstellung entstehen.

Gleitkomma über die Naht

Problem	Wirkung	
0,1 nicht darstellbar	Rundung	**Band 21, Kap. 8**
JSON kennt nur `double`	**2^{53} Grenze**	**große Kennungen**
Textausgabe rundet	Stellen verloren	
Verschiedene Rundungsregeln	0,5 **auf oder ab**	je Sprache

Die zweite Zeile trifft häufiger, als man denkt: Eine Kennung mit neunzehn Stellen passt in ein `int64` und nicht in eine JSON-Zahl. Viele Umsetzungen wandeln sie stillschweigend in `double` — und die letzten Stellen sind fort.

Die Abhilfe ist so einfach wie unbeliebt: Kennungen werden als Zeichenkette übertragen. Sie werden nie berechnet, also verliert man nichts — und gewinnt Eindeutigkeit.

Rundung ist eine fachliche Entscheidung

Regel bei 0,5	**Ergebnis**	
Kaufmännisch	**1**	**verbreitet**
Zur geraden Zahl	**0**	IEEE-Voreinstellung
Immer abwärts	0	
Nach 10^6 Werten	Abweichung	**systematisch**

Die ersten drei Zeilen liefern verschiedene Ergebnisse, und alle drei sind zulässig. Kaufmännisches Runden verschiebt Mittelwerte systematisch nach oben; die IEEE-Regel tut das nicht. Bei einer Million Messwerten ist der Unterschied messbar.

Deshalb gehört die Rundungsregel in den Vertrag — und die Rundung selbst an genau eine Stelle. Wer an drei Stellen rundet, rundet dreimal, und das Ergebnis hängt von der Reihenfolge ab.

Übungsaufgaben

(a) Legen Sie Plausibilitätsgrenzen fest.

(b) Rechnen Sie Einheiten um.

(c) Bestimmen Sie, wo gerundet wird.

Lösung

(a) Plausibilitätsgrenzen.

Größe	**Referenz**	**Kritisch**	**Möglich**
Kalium	3,5–5,1	$< 2{,}5$, $> 6{,}5$	**1–10**
Natrium	135–145	< 120, > 160	90–190
Glukose	70–100	< 40, > 400	10–1500
Temperatur	36–37,5	< 35, > 40	**20–45**

Drei Stufen, drei Zwecke. Der Referenzbereich sagt "auffällig", der kritische Bereich "sofort handeln", und die letzte Spalte sagt "das kann kein Messwert sein". Nur die letzte ist eine Prüfung an der Naht.

Bei Überschreitung	Vorgehen	
Referenzbereich	**Warnung**	fachlich
Kritisch	**Sofort-meldung**	
Möglich	**zurückweisen**	**Fehler, kein Befund**

Die dritte Zeile beseitigt die Fehlerklasse aus dem Vorwort. Ein Kalium von 51 liegt außerhalb des Möglichen — der Wert wird nicht gespeichert, sondern gemeldet, und zwar als technischer Fehler an den Betrieb, nicht als Befund an die Station.

Der Aufwand: eine Tabelle mit drei Spalten je Messgröße, gepflegt vom Labor. Etwa vierhundert Zeilen Daten für ein vollständiges Repertoire — und keine Zeile Code über die Prüfung hinaus.

(b) Einheiten umrechnen.

```
Glukose:  mg/dl -> mmol/l
Faktor = 1 / (Molmasse / 10) = 1 / 18,016
100 mg/dl = 5,55 mmol/l
```

Eigenschaft	Folge	
Faktor stoffabhängig	**je Messgröße**	keine allgemeine Regel
Nicht ganzzahlig	**Rundung**	
Nicht umkehrbar	**hin und zurück ≠ Original**	**Stellen verloren**
Genauigkeit ändert sich	**mitzuführen**	

Die dritte Zeile ist der Grund für die wichtigste Regel dieses Abschnitts: Der ursprüngliche Wert wird gespeichert, wie er gemessen wurde. Umgerechnet wird bei der Anzeige — nie beim Speichern.

Wer beim Eintreffen umrechnet, hat den Originalwert verloren. Wenn sich später herausstellt, dass der Faktor falsch war oder eine andere Einheit gewünscht wird, ist der Bestand nicht mehr zu retten.

(c) Wo gerundet wird.

Stelle	Runden?	
Messgerät	**ja**	**Messgenauigkeit**
Übertragung	**nie**	nur durchreichen
Speicherung	**nie**	wie gemessen
Rechnung	nein	volle Genauigkeit
Anzeige	**ja**	**einmal, am Ende**

Zweimal runden, am Anfang und am Ende — dazwischen nie. Das ist eine der wenigen Regeln dieses Bandes, die ohne Ausnahme gilt, und sie wird in gewachsenen Systemen regelmäßig an vier Stellen verletzt.

Was mehrfaches Runden anrichtet.

Schritte	Ergebnis	Abweichung
Einmal am Ende	**4,83**	—
Nach jedem Schritt	4,80	0,6 %
Bei 10 Schritten	**4,71**	**2,5 %**

Zweieinhalb Prozent Abweichung durch nichts als Rundung an den falschen Stellen. Bei einem Kaliumwert nahe der Grenze entscheidet das darüber, ob eine Warnung erscheint — und niemand kann es später nachvollziehen.

Die Genauigkeit ist eine Aussage

Angabe	Bedeutet	
4,2	**eine Nachkommastelle**	so gemessen
4,20	**zwei**	genauer gemessen
4,2000	vier	**Genauigkeit erfunden**
4,2 als `double`	**unbekannt**	Information fort

Die dritte Zeile entsteht, wenn ein Programm eine feste Formatierung anwendet: Aus 4,2 wird 4,2000, und der Leser hält den Wert für genauer, als er ist. Das ist keine Formatierungsfrage, sondern eine fachliche Falschaussage.

Die vierte Zeile ist der Grund für den Verbund aus Betrag und Exponent: Er trägt die Genauigkeit mit, ein `double` nicht. Band 21, Kapitel 8 hat das begründet; hier ist es eine Anforderung an die Naht.

Die Gleitkommanorm[13] legt Darstellung und Rundungsregeln fest; Goldberg[10] erklärt ihre Folgen; Wirth[26] erinnert daran, dass die Wahl der Darstellung vor der Rechnung kommt.

Teil III

Leistung im Großen

Kapitel 7

Messen im Verbund

Der Mittelwert lügt

Kennzahl	Wert	Sagt
Mittelwert	**42** ms	**fast nichts**
Median	31 ms	**die Hälfte ist schneller**
95. Perzentil	180 ms	
99. Perzentil	**940** ms	**jeder Hundertste**
Maximum	**8,2 s**	einer

Dieselbe Messreihe, fünf Zahlen. Der Mittelwert von zweiundvierzig Millisekunden verdeckt, dass jede hundertste Anfrage fast eine Sekunde braucht — und in einer Klinik ist das die Anfrage, bei der jemand vor dem Bildschirm wartet.

Warum die Verteilung schief ist.

Ursache	Wirkung	
Zwischenspeicher verfehlt	×10	**Kap. 9**
Speicher-bereinigung	×20	**Band 22, Kap. 12**
Netzwiederholung	**×200**	**Zeitablauf**
Wartung, Neustart	×∞	
Ungewöhnlich große Anfrage	×50	**Langzeitpatient**

Fünf Ursachen, die alle selten sind und alle Faktoren im zweistelligen Bereich erzeugen. Deshalb ist die Antwortzeitverteilung nie glockenförmig, und deshalb ist der Mittelwert die falsche Kennzahl.

Warum sich Ausreißer aufschaukeln

Dienste in Kette	99. je Dienst	Anteil langsam
1	940 ms	**1** %
5	940 ms	**4,9** %
20	940 ms	**18** %
100	940 ms	**63** %

Wenn eine Anfrage zwanzig Dienste durchläuft und jeder in einem Prozent der Fälle langsam ist, ist fast jede fünfte Anfrage langsam. Das ist reine Wahrscheinlichkeitsrechnung — und der stärkste Einwand gegen viele kleine Bestandteile.

Daraus folgt: Je mehr Nähte eine Anfrage durchläuft, desto strenger muss das Perzentil jedes einzelnen Dienstes sein. Bei zwanzig Diensten und einer Zielvorgabe von einem Prozent braucht jeder Dienst 99,95 Prozent.

Wo gemessen wird

Messpunkt	Sieht	
Im Dienst	**eigene Arbeit**	**nicht das Warten**
Am Eingang des Dienstes	**plus Warteschlange**	besser
Beim Aufrufer	**plus Netz**	**ehrlicher**
Beim Benutzer	**alles**	**die einzige, die zählt**

Die vier Messpunkte liefern verschiedene Zahlen, und die erste ist immer die schönste. Ein Dienst, der "drei Millisekunden" meldet, während der Benutzer zwei Sekunden wartet, hat recht und misst das Falsche.

Die Differenz zeigt, wo es klemmt.

Messpunkt	95. Perz.	Differenz	
Im Dienst	**18** ms	—	
Am Eingang	47 ms	**29** ms	**Warte-schlange**
Beim Aufrufer	63 ms	16 ms	**Netz**
Beim Benutzer	**410** ms	**347 ms**	**andere Aufrufe**

Die letzte Zeile ist der Befund: Dreihundertsiebenundvierzig Millisekunden entstehen zwischen dem Aufrufer und dem Benutzer — also in anderen Aufrufen derselben Seite. Der untersuchte Dienst war nie das Problem.

Perzentile lassen sich nicht mitteln

Vorgehen	Zulässig?	
Perzentile addieren	**nein**	
Perzentile mitteln	**nein**	**häufiger Fehler**
Über Zeiträume mitteln	nein	
Verteilungen zusammenfassen	**ja**	**Histogramme**

Das 95. Perzentil von zehn Minuten ist nicht der Mittelwert der zehn Minutenwerte. Wer so rechnet, bekommt eine Zahl, die keiner Größe entspricht — und die systematisch zu niedrig ist.

Die richtige Bauform speichert Histogramme statt Perzentile: Zählwerte je Zeitklasse lassen sich addieren, und das Perzentil wird beim Auswerten daraus berechnet. Der Aufwand ist derselbe, das Ergebnis richtig.

Übungsaufgaben

(a) Legen Sie die Kennzahlen fest.

(b) Bestimmen Sie Ziele je Bestandteil.

(c) Untersuchen Sie, was eine Last aussagt.

Lösung

(a) Die Kennzahlen.

Größe	Was sie beantwortet	
Durchsatz	wie viel geht durch	
Antwortzeit	**wie lange wartet man**	Verteilung
Fehlerquote	wie viel schlägt fehl	je Ursache
Sättigung	**wie nah an der Grenze**	**Frühwarnung**
Rückstand	wie viel liegt an	Warteschlangen

Die vierte Zeile ist die einzige, die vorwärts schaut: Durchsatz, Antwortzeit und Fehler melden, dass etwas geschehen ist. Die Sättigung — wie viele Verbindungen belegt, wie voll die Warteschlange, wie viel Speicher — meldet, dass etwas geschehen wird.

Bei einem Klinikbetrieb heißt das konkret: Wenn der Übertragungspuffer regelmäßig zu achtzig Prozent gefüllt ist, weiß man drei Wochen vor dem ersten Datenverlust, dass er kommt.

(b) Ziele je Bestandteil.

Bestandteil	95. Perz.	99. Perz.	Verfügbar
Erfassung	**2** ms	**2** ms	99,99 %
Übertragung	50 ms	200 ms	99,9 %
Speicherung	20 ms	100 ms	**99,95** %
Auswertung	300 ms	**1** s	99,5 %
Ganze Kette	**400** ms	**1,3** s	**99,3** %

Die erste Zeile ist die harte Schranke aus Band 21 — dort steht bei beiden Perzentilen dieselbe Zahl, weil eine harte Schranke keine Ausreißer kennt. Bei allen anderen unterscheiden sich die Werte um Faktor drei bis vier.

Die letzte Zeile ist eine Rechnung, keine Wahl: Die Verfügbarkeit der Kette ist das Produkt der einzelnen. Vier Bestandteile mit je 99,5 bis 99,99 Prozent ergeben 99,3 — etwa zweieinhalb Tage Ausfall im Jahr.

Verfügbarkeit	Ausfall je Jahr	Ausfall je Monat	
99 %	3,7 d	7,3 h	
99,9 %	8,8 h	44 min	
99,99 %	**53** min	4,4 min	
99,999 %	5,3 min	26 s	teuer

Jede Neun kostet etwa das Dreifache der vorigen. Für eine Übertragungsschicht mit Puffer sind drei Neunen richtig; für die Erfassung am Patientenbett vier — und fünf braucht in einer Klinik fast nichts.

(c) Was eine Last aussagt.

Versuchsaufbau	Aussage-kräftig?	
Ein Wert, wiederholt	**nein**	**alles zwischen-gespeichert**
Zufällige Werte	teils	Zwischen-speicher wirkungslos
Verteilung wie im Betrieb	**ja**	**aufgezeichnete Last**
Nur der Regelfall	nein	**Langzeit-patient fehlt**

Die ersten beiden Zeilen sind die beiden Extreme, und beide liefern falsche Zahlen — die eine zu gut, die andere zu schlecht. Nur eine Last, deren Verteilung dem Betrieb entspricht, misst, was im Betrieb geschieht.

Die billigste Antwort ist eine aufgezeichnete Last: Man protokolliert einen Tag im Betrieb mit und spielt ihn ab. Der Aufwand liegt bei einem Tag Arbeit und ersetzt jede Diskussion über realistische Annahmen.

Messen im Betrieb, nicht im Versuch

Unterschied	Versuch	Betrieb
Daten	klein	gewachsen
Nebenlast	keine	Sicherung, Wartung
Netz	lokal	Haus
Ausreißer	**fehlen**	**da**

Die letzte Zeile ist der Grund, warum Versuchsmessungen die Perzentile systematisch zu gut anzeigen: Genau die seltenen Störungen, die den Ausläufer erzeugen, kommen im Versuch nicht vor. Ein Versuchsaufbau misst den Median gut und das 99. Perzentil gar nicht.

Deshalb gehört die Messung dauerhaft in den Betrieb — nicht als einmalige Untersuchung. Kapitel 11 behandelt, wie man das einrichtet.

Gregg[11] liefert das Vorgehen zur Messung der Leistung im Systemverbund; Dean und Barroso[28] erklären, warum Ausläufer sich über Dienstketten aufschaukeln; das Betriebshandbuch[3] verbindet Kennzahlen mit Zielvorgaben.

Kapitel 8

Wo die Zeit hingeht

Die Aufteilung einer Anfrage

Abschnitt	Anteil	
Datenbank-abfragen	≈ **48** %	**davon** $n{+}1$**:** 32 %
Netz zwischen Diensten	≈ 18 %	
Serialisierung	≈ 12 %	oft übersehen
Eigene Rechnung	≈ 11 %	
Warten auf Sperren	≈ 7 %	
Protokollierung	≈ 4 %	**Band 22, Kap. 12**

Elf Prozent der Zeit vergehen dort, wo die meisten Optimierungsversuche ansetzen. Fast die Hälfte in der Datenbank, und zwei Drittel davon durch ein einziges Muster — dasselbe $n{+}1$-Problem wie in Band 20, Kapitel 12 und Band 23, Kapitel 2.

Die Serialisierung, die niemand rechnet.

Vorgang	**je** 1000	**Bei** 5 **Nähten**	
JSON erzeugen	4,8 ms	**24** ms	
JSON lesen	6,1 ms	**31** ms	
Binär erzeugen	**0,2** ms	1 ms	24×
Binär lesen	0,3 ms	1,5 ms	

Fünfundfünfzig Millisekunden für das Umwandeln derselben tausend Befunde, wenn sie durch fünf Nähte gehen. Das ist mehr, als die Fachlogik braucht — und es steht in keinem Profil eines einzelnen Dienstes.

Die Ablaufverfolgung über Nähte

Das Mittel	Eine Kennung wandert mit der Anfrage durch alle Bestandteile
Was man sieht	**Wo die Zeit vergeht — über Dienstgrenzen hinweg**
Der Aufwand	Kennung durchreichen, Abschnitte melden

Ohne dieses Mittel misst jeder Dienst sich selbst, und niemand sieht das Ganze. Mit ihm bekommt man ein Bild einer einzelnen Anfrage über alle Nähte — und die Frage "warum hat das zwei Sekunden gedauert" wird beantwortbar.

Der Aufwand ist gering und die Disziplin hoch: Die Kennung muss durch jede Naht weitergereicht werden, auch durch Warteschlangen und asynchrone Verarbeitung. Wo sie verlorengeht, endet die Verfolgung.

Das $n+1$-Problem über Nähte

```
// Auswertung fragt die Speicherung
faelle = hole_faelle(patient)       // 1 Aufruf
for fall in faelle:
befunde = hole_befunde(fall.id) // n Aufrufe ueber die Naht
```

Fälle	Im Prozess	Über Naht	Faktor
12	52 ms	**190** ms	3,7
80	324 ms	1,2 s	
340	1,4 s	**5,1 s**	**Langzeit-patient**

Dasselbe Muster wie in Band 20, mit vierfachem Faktor — weil jeder Aufruf jetzt Netz, Serialisierung und Warteschlange durchläuft. Fünf Sekunden für den Aufbau einer Patientenansicht, und das bei dem Patienten, bei dem es am meisten zählt.

Die Behebung.

Weg	Dauer	
Einzeln	**5,1** s	
Alle Fallnummern auf einmal	**78** ms	65×
Ein Aufruf für alles	**61** ms	**fachlicher Zuschnitt**
Nebenläufig einzeln	340 ms	hilft, löst nicht

Die letzte Zeile ist die Versuchung: Nebenläufigkeit bringt Faktor fünfzehn und die Bündelung Faktor fünfundsechzig. Wer nebenläufig einzeln abfragt, hat das Problem verschleiert statt behoben — und die Last auf der Gegenseite verzehnfacht.

Der Zuschnitt der Naht entscheidet

	Technisch geschnitten	Fachlich geschnitten
Aufrufe je Ansicht	**n+1**	**1**
Wiederverwendbar	ja	weniger
Übertragene Menge	genau	etwas mehr
Passt zur Aufgabe	**nein**	**ja**

Eine Naht, die `holeBefund(id)` anbietet, ist allgemein und erzwingt $n+1$. Eine, die `holeVerlauf(patient, groesse, zeitraum)` anbietet, ist enger und beantwortet die tatsächliche Frage in einem Aufruf — Ousterhouts tiefe Schnittstelle aus Band 20, hier über das Netz.

Der Preis steht in der zweiten Zeile und ist geringer, als er wirkt: Eine Naht mit acht fachlichen Operationen deckt fast alles ab, was gebraucht wird. Wer hundert allgemeine Operationen anbietet, hat keine Schnittstelle, sondern eine Datenbank über das Netz.

Übungsaufgaben

(a) Verfolgen Sie eine langsame Anfrage.

(b) Rechnen Sie die Wirkung der Bündelung aus.

(c) Bestimmen Sie, wo man zuerst ansetzt.

Lösung

(a) Eine langsame Anfrage verfolgen.
Die Meldung. Der Aufbau der Verlaufsansicht dauert bei manchen Patienten über fünf Sekunden.

	Abschnitt	Dauer	Anteil
1	Oberfläche	40 ms	1 %
2	Auswertung, Rechnung	180 ms	4 %
3	**Aufrufe an Speicherung**	**4,6 s**	**90 %**
4	davon Netz	1,1 s	
5	davon Serialisierung	**0,9** s	
6	davon Datenbank	2,6 s	

Neunzig Prozent in einem Abschnitt, und die Aufteilung darunter zeigt: Selbst wenn die Datenbank unendlich schnell wäre, blieben zwei Sekunden. Das ist der Befund, den ein Datenbankprofil allein nie liefert.

Die fünfte Zeile ist der überraschende Posten: Neunhundert Millisekunden nur für das Umwandeln in JSON und zurück — bei dreihundertvierzig Einzelaufrufen mit je einem kleinen Ergebnis.

(b) Die Wirkung der Bündelung.

Maßnahme	Netz	Serial.	Datenbank
Vorher	1,1 s	0,9 s	2,6 s
Gebündelt	**4** ms	**31** ms	**43** ms
Faktor	**275**	29	**60**

Eine Änderung, drei Posten behoben. Der größte Faktor liegt beim Netz, weil dort die Verzögerung je Aufruf anfällt und nicht die übertragene Menge — dreihundertvierzig mal drei Millisekunden gegen einmal vier.

	Vorher	Nachher	
Gesamt	**5,1** s	**298** ms	17×
Übertragene Menge	1,2 MB	**1,4** MB	+17 %
Zeilen geändert	—	**24**	
Aufwand	—	**4** h	

Vier Stunden Arbeit, vierundzwanzig geänderte Zeilen, Faktor siebzehn. Die zweite Zeile ist der Preis: siebzehn Prozent mehr Daten, weil der gebündelte Aufruf etwas mitliefert, was nicht jeder braucht.

(c) Wo man zuerst ansetzt.

	Schritt	Aufwand	Gewinn
1	**Verfolgen, nicht raten**	2 h	—
2	**Aufrufe über Nähte bündeln**	1 d	5–60×
3	Datenbankindizes	1 d	2–50×
4	Zwischenspeicher	2 d	2–20×
5	Format wechseln	3 d	2–5×
6	Mehr Rechner	laufend	1–2×

Die Reihenfolge ist nach dem Verhältnis von Gewinn zu Aufwand geordnet, und sie wird regelmäßig von unten begonnen. Schritt sechs ist der teuerste und der schwächste — doppelte Rechnerzahl bringt bei einem $n+1$-Problem gar nichts, weil die Verzögerung nicht von der Rechenleistung abhängt.

Schritt zwei steht vorn, weil er in einem verteilten Werk fast immer trägt — und weil er die einzige Maßnahme ist, die drei Kostenposten zugleich senkt: Netz, Serialisierung und Datenbank.

Die Verzögerung addiert sich, die Bandbreite nicht

	Verzögerung	Bandbreite
Bei mehr Aufrufen	**addiert sich**	unverändert
Bei mehr Daten	fast unverändert	steigt
Durch schnellere Leitung	kaum besser	**besser**
Durch kürzere Strecke	**besser**	unverändert

Die dritte Zeile überrascht regelmäßig: Eine zehnmal schnellere Leitung verkürzt die Verzögerung fast nicht, weil sie von der Laufzeit des Signals und der Zahl der Zwischenstationen bestimmt wird. Bandbreite kauft man, Verzögerung nicht.

Daraus folgt die praktische Regel: Wenige große Aufrufe schlagen viele kleine, auch wenn dabei mehr Daten übertragen werden. Siebzehn Prozent mehr Menge gegen Faktor siebzehn Zeit — das ist der Handel aus Aufgabe (b).

Gregg[11] zeigt, wie man Abschnitte einer Anfrage trennt; Fowler[8] sammelt die Bauformen, die $n{+}1$ vermeiden; Kleppmann[15] ordnet die Abwägungen zwischen Menge und Verzögerung ein.

Kapitel 9

Zwischenspeicher

Der Handel

Man bekommt	Man bezahlt	
Geschwindigkeit	**Aktualität**	**der ganze Handel**
Entlastung	Speicher	
Verfügbarkeit	Komplexität	Ungültigmachen

Die erste Zeile ist der ganze Begriff: Ein Zwischenspeicher liefert schnell und möglicherweise veraltet. Die Frage ist nie, ob er veraltet, sondern wie lange und ob das erträglich ist.

Was erträglich ist, hängt vom Gegenstand ab.

Gegenstand	Erträglich veraltet	
Messgrößen-katalog	**Stunden**	ändert sich selten
Referenzbereiche	**Minuten**	Band 21, Kap. 11
Patienten-stammdaten	Minuten	
Befunde	Sekunden	
Kritische Warnung	**gar nicht**	**nie zwischen-speichern**

Die letzte Zeile ist eine fachliche Festlegung, keine technische: Eine kritische Warnung darf nicht aus einem Zwischenspeicher kommen, weil ein veralteter Wert hier unmittelbar schadet. Wer alles zwischenspeichert, hat diese Unterscheidung nicht getroffen.

Die drei Arten, ungültig zu machen

Art	Aufwand	Veralterung
Ablaufzeit	**sehr gering**	**bis zur Frist**
Bei Änderung löschen	mittel	kurz
Bei Änderung neu füllen	hoch	**keine**

Die erste Zeile ist fast immer die richtige Wahl und wird zugunsten der beiden anderen verworfen, weil sie zu einfach wirkt. Eine Ablaufzeit braucht keine Benachrichtigung, keine Reihenfolge und keine Behandlung verlorener Nachrichten — sie kann nur zu lang gewählt sein.

Die zweite und dritte Zeile setzen voraus, dass jede Änderung zuverlässig gemeldet wird. Über eine Naht ist das die Annahme aus Kapitel 2, die nicht zutrifft — eine verlorene Ungültigmeldung erzeugt einen Zwischenspeicher, der dauerhaft falsch ist.

Wo der Zwischenspeicher liegt

Ort	Zugriff	Ungültig machen	Kohärent
Im Prozess	**0,1** µs	**schwer**	**nein**
Gemeinsam im Haus	200 µs	leicht	ja
Beim Aufrufer	**0,1** µs	sehr schwer	nein
Datenbankseitig	80 µs	**eingebaut**	ja

Die erste Zeile ist zweitausendmal schneller und nicht kohärent: Bei zehn Prozessen gibt es zehn Zwischenspeicher, die verschiedene Werte halten können. Ein Arzt sieht 5,1, sein Kollege gleichzeitig 4,8 — und beide Bildschirme stimmen mit ihrem Prozess überein.

Deshalb gilt: Prozesseigene Zwischenspeicher nur für Dinge, die sich fast nie ändern — Katalog, Codelisten, Konfiguration. Alles, was ein Benutzer als "aktuell" erwartet, gehört in einen gemeinsamen.

Was bei leerem Zwischenspeicher geschieht

Lage	Last dahinter	
Normalbetrieb, 95 % Treffer	**5** %	
Nach Neustart	**100** %	×20
Alle Einträge zugleich abgelaufen	**100** %	**Gleichtakt**
Beliebter Eintrag fehlt	**Ansturm**	***n* gleichzeitige Anfragen**

Ein System, das mit fünfundneunzig Prozent Trefferquote läuft, ist auf fünf Prozent der Last ausgelegt. Nach einem Neustart trifft es die zwanzigfache Last — und wenn es die nicht verkraftet, kommt es nicht wieder hoch.

Die dritte Zeile ist vermeidbar: Wer alle Ablaufzeiten um einen zufälligen Betrag streut, verhindert, dass sie gleichzeitig ablaufen. Fünf Zeilen, und ein wiederkehrender Lastberg verschwindet.

Übungsaufgaben

(a) Legen Sie die Zwischenspeicherung fest.

(b) Verhindern Sie den Ansturm.

(c) Bestimmen Sie, wann Veralterung erträglich ist.

Lösung

(a) Die Festlegung.

Gegenstand	Ort	Frist	Trefferquote
Messgrößen	**Prozess**	1 h	**99,9** %
Referenzbereiche	Prozess	**5** min	99,5 %
Stammdaten	**gemeinsam**	2 min	87 %
Verläufe	gemeinsam	**30** s	41 %
Warnungen	**keiner**	—	—

Die vierte Zeile lohnt kaum: Bei einundvierzig Prozent Trefferquote spart der Zwischenspeicher weniger als die Hälfte der Last und bringt die volle Komplexität mit. Eine Trefferquote unter etwa sechzig Prozent ist ein Grund, es zu lassen.

	Ohne	Mit
Last auf Speicherung	**100** %	**18** %
95. Perzentil	410 ms	**95** ms
Veralterung möglich	nein	bis 2 min
Zusätzliche Zeilen	0	≈ 160

Die dritte Zeile ist die Frage, die fachlich zu beantworten ist: Sind zwei Minuten veraltete Stammdaten erträglich? Für die Anzeige des Patientennamens ja, für die Prüfung einer Medikamentenunverträglichkeit nicht.

(b) Den Ansturm verhindern.

Mittel	Aufwand	
Ablaufzeiten streuen	5 **Zeilen**	gegen Gleichtakt
Nur einer füllt nach	gering	**gegen Ansturm**
Alten Wert weiterliefern	mittel	während des Nachfüllens
Vorwärmen beim Start	mittel	gegen Neustartlast
Last begrenzen	gering	**Band 22, Kap. 4**

Die zweite Zeile ist der Kunstgriff: Wenn hundert Anfragen gleichzeitig denselben fehlenden Eintrag brauchen, holt ihn eine und die anderen warten auf das Ergebnis. Ohne diese Sperre stellen alle hundert dieselbe Anfrage an die Speicherung.

	Ohne	Mit
Anfragen nach hinten	**100**	**1**
Dauer	**2,4** s	**51** ms
Last auf Speicherung	Spitze	flach

Faktor siebenundvierzig, und die Ursache ist nicht Geschwindigkeit, sondern vermiedene Arbeit. Die dritte Zeile ist der eigentliche Gewinn: Die Speicherung sieht eine flache Last statt Spitzen, und ihre eigenen Perzentile bleiben stabil.

(c) Wann Veralterung erträglich ist.

Prüffrage	Antwort	
Was geschieht bei 2 min alt?	**fachlich prüfen**	
Merkt es jemand?	**oft nein**	
Schadet es jemandem?	**entscheidend**	
Ist es nachvollziehbar?	**muss ja sein**	**Kap. 4**

Die letzte Zeile wird selten gestellt und ist regulatorisch die wichtigste: Wenn ein Arzt eine Entscheidung auf Grundlage veralteter Daten trifft, muss später feststellbar sein, welchen Stand er gesehen hat. Das ist die Systemzeit aus Kapitel 4 — und ein Zwischenspeicher, der sie nicht mitführt, macht die Frage unbeantwortbar.

Die praktische Folge: Jede zwischengespeicherte Antwort trägt den Zeitpunkt, zu dem sie erhoben wurde. Ein Feld mehr, und die Anzeige kann "Stand: 14:32" zeigen — was den Arzt in die Lage versetzt, selbst zu entscheiden, ob ihm das reicht.

Der Zwischenspeicher ist ein zweiter Datenbestand

Frage	Beantwortet?	
Wer darf es lesen?	**oft nicht geprüft**	**Band 19**
Wird es verschlüsselt?	selten	
Steht es in der Löschfrist?	**fast nie**	**Patienten-daten**
Wird es gesichert?	muss nicht	ableitbar

Die dritte Zeile ist ein regelmäßiger Befund bei Prüfungen: Patientendaten liegen in einem Zwischenspeicher, der in keinem Löschkonzept auftaucht — weil ihn niemand für einen Datenbestand hält. Er ist einer, mit denselben Pflichten.

Die erste Zeile ist die gefährlichste: Wenn die Rechteprüfung vor dem Zwischenspeicher liegt und der Schlüssel die Benutzerkennung nicht enthält, sieht der zweite Benutzer die Daten des ersten. Ein Fehler, der bei jeder Einführung eines Zwischenspeichers zu prüfen ist.

Kleppmann[15] ordnet Zwischenspeicherung und Kohärenz ein; Vogels[24] beschreibt, was verzögerte Konsistenz praktisch bedeutet; Brewer[4] liefert den Rahmen, in dem diese Abwägung steht.

Teil IV

Der Betrieb

Kapitel 10

Fehler im Verbund

Wiederholen ist gefährlich

Lage	Wiederholen?	
Verbindung nicht zustande	**ja**	**sicher nicht angekommen**
Fehler 5*xx*	ja	wenn idempotent
Zeitablauf	**nur wenn idempotent**	**unklar**
Fehler 4*xx*	**nein**	es bleibt falsch
Zurückweisung wegen Last	ja	**mit Wartezeit**

Die vierte Zeile wird am häufigsten verletzt: Eine automatische Wiederholung bei einem Formatfehler erzeugt endlose Versuche mit derselben falschen Nachricht — und belastet den Empfänger, ohne je zum Ziel zu kommen.

Wie oft und wie schnell.

Verfahren	Abstände	
Sofort, unbegrenzt	0, 0, 0	**Lawine**
Fester Abstand	1 s, 1 s, 1 s	Gleichtakt
Verdoppelnd	1, 2, 4, 8 s	besser
Verdoppelnd mit Streuung	1, 1,7, 4,4, 9,1	**richtig**

Die letzte Zeile ist der Standard und wird selten vollständig umgesetzt: Die Streuung verhindert, dass tausend Geräte nach einem Ausfall gleichzeitig wiederholen — und genau das bringt den Empfänger beim Wiederanlauf ein zweites Mal zu Fall.

Der Sicherungsschalter

Zustand	Verhalten	Übergang
Geschlossen	durchlassen	**bei n Fehlern → offen**
Offen	**sofort ablehnen**	nach t → halboffen
Halboffen	**eine Anfrage durch**	Erfolg → geschlossen

Der Zweck ist nicht, den Ausfall zu verhindern — der ist schon da. Der Zweck ist, den Aufrufer nicht mit hineinzuziehen: Ohne Schalter warten hundert Anfragen je dreißig Sekunden auf einen toten Dienst und blockieren dabei den Aufrufer.

Die dritte Zeile ist die feine Stelle: Genau eine Anfrage wird durchgelassen, um zu prüfen, ob der Dienst zurück ist. Wer alle durchlässt, bringt den gerade wiederanlaufenden Dienst sofort erneut zu Fall.

Zeitschranken

Ohne Zeitschranke	Folge	
Aufruf hängt	**Faden belegt**	
Weitere Aufrufe	Warteschlange wächst	
Speicher	**läuft voll**	
Aufrufer	**fällt ebenfalls aus**	**Kaskade**

Vier Schritte von einem hängenden Aufruf zum Ausfall des Aufrufers. Das ist der häufigste Weg, auf dem ein Teilausfall zu einem Gesamtausfall wird — und er ist mit einer Zeitschranke je Aufruf zu verhindern.

Wie lang?

Grundlage	Wert	
Geraten	30 s	**viel zu lang**
Mittelwert ×2	84 ms	zu kurz
99. **Perzentil** ×1,5	**1,4 s**	**begründet**
Restzeit aus dem Aufruf	dynamisch	am besten

Die erste Zeile ist die Vorgabe vieler Bibliotheken und in der Klinik unbrauchbar: Dreißig Sekunden Warten bedeuten, dass der Benutzer längst aufgegeben hat und der Faden trotzdem belegt bleibt. Die Zahl muss aus der Messung kommen.

Die Frist muss durch die Kette

Ebene	Frist	Verbraucht	Rest
Benutzer	**2000** ms	—	2000
Auswertung	1900 ms	120	1780
Speicherung	1700 ms	340	**1440**
Datenbank	**1400** ms		

Jede Ebene gibt weniger Zeit weiter, als sie selbst hat — die verbrauchte abgezogen und eine Spanne für die Antwort. Ohne diese Weitergabe wartet die Datenbank noch, wenn der Benutzer längst eine Fehlermeldung sieht: Arbeit ohne Empfänger.

Das ist derselbe Abbruchkontext wie in Band 22, Kapitel 4 — hier über Prozessgrenzen. Er muss durch jede Naht weitergereicht werden, und wo er verlorengeht, arbeitet ein Dienst für niemanden.

Übungsaufgaben

(a) Verfolgen Sie eine Kaskade.

(b) Bestimmen Sie das Verhalten bei Überlast.

(c) Legen Sie fest, was bei endgültigem Fehlschlag geschieht.

Lösung

(a) Eine Kaskade verfolgen.

Zeit	Ereignis	Wirkung
t_0	Datenbank wird langsam	Antwortzeit $\times 8$
t_1	**Speicherdienst: Fäden belegt**	Warteschlange wächst
t_2	Auswertung: Zeitabläufe	**wiederholt**
t_3	**Last verdreifacht**	**durch Wiederholung**
t_4	Speicherdienst fällt aus	
t_5	Auswertung fällt aus	**Gesamtausfall**

Sechs Schritte von einer langsamen Datenbank zum Gesamtausfall, und der entscheidende ist der dritte: Die Wiederholungen verdreifachen die Last auf ein System, das schon überlastet ist. Ohne sie wäre es bei langsam geblieben.

Was jede Maßnahme verhindert hätte.

Maßnahme	Stoppt bei	
Zeitschranken	t_1	Fäden frei
Nebenläufigkeit begrenzt	t_1	**Band 22, Kap. 4**
Wiederholungen begrenzt	t_3	**keine Verdreifachung**
Sicherungs-schalter	t_4	Auswertung überlebt
Last abweisen	t_2	**Teilbetrieb**

Fünf Maßnahmen, jede hätte allein genügt. Das ist der Regelfall bei Kaskaden — sie brauchen eine Kette von Versäumnissen, und jedes einzelne Glied bricht sie.

(b) Verhalten bei Überlast.

Vorgehen	Ergebnis	
Alles annehmen	**alle werden langsam**	dann Ausfall
Überschuss abweisen	**die meisten normal**	**richtig**
Nach Wichtigkeit	**Warnungen zuerst**	besser
Warteschlange verlängern	verzögert	Kap. 7

Die zweite Zeile ist die unbequeme Einsicht: Bei Überlast ist es besser, zehn Prozent der Anfragen sofort abzuweisen, als alle langsam zu bedienen. Eine abgewiesene Anfrage kann der Aufrufer wiederholen; eine, die nach zwanzig Sekunden antwortet, hilft niemandem mehr.

Die dritte Zeile ist in der Klinik die richtige Verfeinerung: Kritische Warnungen werden immer bedient, Verlaufsansichten bei Überlast abgewiesen. Das setzt voraus, dass die Naht die Wichtigkeit überträgt — ein Feld, das man vorsehen muss.

(c) Endgültiger Fehlschlag.

Vorgehen	Tauglich?	
Verwerfen	**nie**	**Datenverlust**
Ewig wiederholen	nein	blockiert die Reihe
In Sonderablage	**ja**	**mit Meldung**
Manuell nachbearbeiten	ja	**jemand muss zuständig sein**

Die zweite Zeile ist ein häufiger Entwurfsfehler bei Warteschlangen: Eine Nachricht, die nie verarbeitet werden kann, blockiert alle dahinter — weil die Reihenfolge gewahrt bleiben soll. Nach drei Versuchen gehört sie beiseite.

Anforderung an die Ablage	Nötig?	
Vollständige Nachricht	**ja**	wiederholbar
Fehlergrund	ja	
Zeitpunkt und Versuche	ja	
Wird angesehen	**ja**	**sonst nutzlos**
Zählwert überwacht	ja	Kap. 11

Die vierte Zeile ist keine technische Anforderung, sondern eine organisatorische — und sie entscheidet über den Wert der ganzen Bauform. Eine Sonderablage ohne Zuständigen ist ein Ort, an dem Befunde unbemerkt liegenbleiben.

Der Fehler muss den Benutzer erreichen

Anzeige	Tauglich?	
“Ein Fehler ist aufgetreten”	**nein**	keine Handlung ableitbar
“Befunde ab 14:20 fehlen”	**ja**	**Umfang klar**
Nichts anzeigen	**gefährlich**	**sieht vollständig aus**
Kennung zum Nachfragen	ja	

Die dritte Zeile ist die gefährlichste Bauform überhaupt: Eine Verlaufsansicht, der Werte fehlen, ohne dass es angezeigt wird. Der Arzt sieht eine vollständige Kurve und trifft eine Entscheidung auf unvollständiger Grundlage.

Die Regel für klinische Anzeigen lautet deshalb: Unvollständigkeit wird immer sichtbar gemacht. Lieber eine Ansicht mit dem Hinweis “Zeitraum 14:20 bis 14:50 konnte nicht geladen werden” als eine scheinbar vollständige.

Nygard[18] sammelt die Bauformen gegen Kaskaden; Postels Regel[20] legt die Strenge je Seite fest; Richardson[21] zeigt die Umsetzung in verteilten Diensten.

Kapitel 11

Beobachtbarkeit

Drei Arten von Aufzeichnung

	Protokolle	Zählwerte	Verfolgung
Antwortet	**was geschah**	**wie viel**	**wo die Zeit**
Menge	groß	klein	Stichprobe
Kosten	hoch	gering	mittel
Einzelfall	**ja**	nein	**ja**
Trend	mühsam	**ja**	nein

Drei Werkzeuge für drei Fragen, und keines ersetzt die anderen. Wer nur Protokolle hat, kann keinen Trend erkennen; wer nur Zählwerte hat, kann keinen Einzelfall aufklären.

Die Frage entscheidet über das Mittel.

Frage	Mittel	
Läuft es?	**Zählwerte**	Warnung
Wird es schlechter?	Zählwerte	Trend
Warum war diese Anfrage langsam?	**Verfolgung**	Kap. 8
Was ist bei Patient X passiert?	**Protokolle**	**Einzelfall**
Wer hat wann was gesehen?	**Protokolle**	**regulatorisch**

Die letzte Zeile ist in der Klinik keine Betriebsfrage, sondern eine Pflicht — und sie verlangt ein Protokoll mit anderen Anforderungen: unveränderlich, vollständig, aufbewahrungspflichtig. Es ist ein eigener Bestand, kein Betriebsprotokoll.

Die zwei Protokolle

	Betrieb	Nachweis
Zweck	**Fehlersuche**	**wer sah was**
Aufbewahrung	7–30 Tage	**Jahre**
Veränderbar	ja	**nein**
Patientendaten	**keine**	Kennungen
Wer liest	Entwickler	Prüfer, Gericht

Die vierte Zeile ist die Regel aus Band 19 und Band 21: In ein Betriebsprotokoll gehören Kennungen, keine Inhalte. Es wandert in Auswertungssysteme mit anderen Zugriffsrechten — und ein Kaliumwert in einer Protokollzeile ist eine Offenlegung.

Was ein Zählwert braucht

Eigenschaft	Warum	
Merkmale (Dienst, Art)	**aufschlüsselbar**	
Wenige Merkmalswerte	**Zahl der Reihen**	**nie Patientenkennung**
Als Histogramm	**Perzentile**	Kap. 7
Monoton steigend	**Neustart erkennbar**	

Die zweite Zeile ist die häufigste Kostenfalle: Ein Zählwert mit der Patientenkennung als Merkmal erzeugt eine Reihe je Patient — bei zweihunderttausend Patienten zweihunderttausend Reihen, und das Überwachungssystem bricht zusammen.

Die vier, die immer gebraucht werden.

Zählwert	Warnt bei	
Anfragen je Sekunde	**Einbruch**	**oft der erste Hinweis**
Fehlerquote	Anstieg	
Antwortzeitverteilung	99. Perzentil	
Sättigung	$> 70\,\%$	**Vorwarnung**

Die erste Zeile wird oft übersehen: Ein Einbruch der Anfragezahl ist ein Störungshinweis — irgendwer vorne fragt nicht mehr. Wer nur auf Fehler achtet, merkt einen ausgefallenen Sender gar nicht, weil er keine Fehler erzeugt.

Alarme, die niemand mehr liest

Eigenschaft	Regel	
Löst aus bei	**Symptom, nicht Ursache**	Benutzer betroffen
Handlung ableitbar	**muss**	sonst weglassen
Fehlalarme	< 10 %	**Band 18**
Nachts wecken	nur wenn nötig	

Die erste Zeile ist die wichtigste Umstellung: Nicht "Speicherbelegung über achtzig Prozent" melden, sondern "Befunde erreichen die Station verspätet". Das erste ist eine Vermutung über eine Ursache, das zweite ist der Schaden.

Dieselbe Alarmermüdung wie bei den klinischen Geräten aus Band 18: Bei mehr als zehn Prozent Fehlalarmen wird jeder Alarm zum Rauschen — und dann auch der echte.

Übungsaufgaben

(a) Legen Sie die Aufzeichnung je Bestandteil fest.

(b) Bestimmen Sie die Alarme.

(c) Untersuchen Sie die Kosten.

Lösung

(a) Aufzeichnung je Bestandteil.

Bestandteil	Protokoll	Zählwerte	Verfolgung
Erfassung	knapp	6	nein
Übertragung	voll	**14**	**ja**
Speicherung	voll	11	ja
Auswertung	voll	9	ja

Die erste Zeile folgt aus Band 21: Der Messdatenerfasser hat vierundsechzig Kilobyte und kann keine Verfolgung mitführen. Er zählt — gemessen, gesendet, verworfen, Puffer voll, Fehler, Neustarts — und das genügt.

Die zweite Zeile ist der Ort, an dem die Verfolgung beginnt: Die Übertragungsschicht vergibt die Kennung und reicht sie weiter. Damit ist jeder Messwert vom Eintritt in das Werk bis zur Anzeige verfolgbar.

(b) Die Alarme.

Alarm	Dringend?	Bedeutet
Keine Befunde seit 10 min	**ja**	**Kette unterbrochen**
Fehlerquote > 1 %	ja	
99. Perzentil > 2 s	tagsüber	
Sonderablage wächst	tagsüber	Kap. 10
Puffer > 70 %	tagsüber	**Vorwarnung**
Rechner ausgelastet	**nein**	**keine Handlung**

Die letzte Zeile ist der Alarm, den fast jedes Überwachungssystem voreingestellt mitbringt und der fast nie etwas bedeutet: Ein ausgelasteter Rechner, der seine Antwortzeiten hält, arbeitet wie vorgesehen. Er gehört in ein Schaubild, nicht in einen Alarm.

Die erste Zeile ist der wichtigste Alarm des ganzen Werks und der am seltensten eingerichtete: Wenn keine Befunde mehr ankommen, ist die Kette irgendwo unterbrochen — und niemand erzeugt dabei einen Fehler.

(c) Die Kosten.

Art	Menge/Tag	Speicher	Laufzeit
Protokolle, alles	**240** GB	teuer	8 %
Protokolle, gefiltert	**12** GB	tragbar	2 %
Zählwerte	**80** MB	**gering**	< 0,5 %
Verfolgung, 1 %	**1,4** GB	gering	1 %

Die dritte Zeile ist das beste Verhältnis: Achtzig Megabyte je Tag beantworten alle Fragen nach Trend und Zustand. Die erste Zeile ist dreitausendmal so groß und beantwortet Fragen, die man selten stellt.

Was gefiltert wird	Anteil	
Erfolgreiche Routine	**94** %	**Zählwert genügt**
Fehler	**100** % behalten	
Langsame Anfragen	100 % behalten	über Schwelle
Stichprobe der Routine	**1** %	für Vergleiche

Faktor zwanzig durch eine einfache Regel: Alles Fehlerhafte und alles Langsame vollständig, vom Rest ein Prozent. Damit bleibt jede interessante Anfrage im Protokoll und die uninteressante Mehrheit verschwindet.

Die Aufzeichnung gehört zum Entwurf

Zeitpunkt	Aufwand	
Beim Entwurf	≈ 3 %	**eingebaut**
Nach der Fertigstellung	≈ 12 %	
Während einer Störung	**unmöglich**	**man braucht sie jetzt**

Die dritte Zeile ist der Grund für dieses Kapitel: Man kann eine Verfolgung nicht einbauen, während man eine Störung untersucht. Entweder sie war da, oder man rät — und Raten ist bei einem Werk aus fünf Bestandteilen aussichtslos.

Drei Prozent des Aufwands beim Entwurf gegen zwölf danach. Das ist dieselbe Rechnung wie bei der Prüfbarkeit in Band 20, Kapitel 9 — und sie wird aus demselben Grund verschoben: Beides sieht aus wie Arbeit, die nichts hervorbringt.

Majors und Mitautorinnen[16] entwickeln den Begriff der Beobachtbarkeit; das Betriebshandbuch[3] verbindet ihn mit Zielen und Alarmen; Gregg[11] liefert die Messtechnik darunter.

Kapitel 12

Ausfall und Wiederanlauf

Was bei einem Ausfall verloren geht

Ort	Verloren	
Im Arbeitsspeicher	**alles**	
In der Warteschlange	nichts	wenn dauerhaft
Auf der Platte, ungeschrieben	**Puffer**	**bis 30 s**
In der Datenbank	nichts	wenn bestätigt

Die dritte Zeile überrascht regelmäßig: Ein geschriebener Datensatz liegt zunächst im Puffer des Betriebssystems, nicht auf der Platte. Bei einem Stromausfall sind bis zu dreißig Sekunden Schreibvorgänge fort — obwohl das Programm sie als erfolgreich gemeldet hat.

Was eine Bestätigung bedeutet.

Bestätigt heißt	Sicher bei	
Angenommen	nichts	**noch im Speicher**
Geschrieben	Programm-absturz	nicht Stromausfall
Durchgeschrieben	**Stromausfall**	**teuer**
Auf zwei Rechnern	Plattenschaden	teurer

Vier Stufen mit vier Preisen. Die dritte kostet etwa Faktor zwanzig gegenüber der zweiten — und für Befunddaten ist sie die richtige Wahl, für ein Betriebsprotokoll nicht.

Die Kette der Zusicherungen

Stelle	Bestätigt wann	
Gerät	**nach Bestätigung**	puffert bis dahin
Übertragung	nach Speicherbestätigung	reicht durch
Speicherung	**nach Durchschreiben**	

Die Zusicherung ist nur so stark wie ihr schwächstes Glied. Wenn die Übertragung bestätigt, bevor die Speicherung geschrieben hat, verspricht sie etwas, das sie nicht halten kann — und das Gerät löscht seinen Puffer.

Die praktische Regel: Ein Bestandteil bestätigt erst, wenn der nächste bestätigt hat. Das kostet Verzögerung und macht die Kette so verlässlich wie ihr Ende.

Der Wiederanlauf

Frage	Antwort nötig	
Wo weitermachen?	**letzter Stand**	dauerhaft merken
Was war halb fertig?	**erkennen**	**häufigste Lücke**
Wie lange dauert es?	**gemessen**	
Verträgt die Gegenseite es?	**Last beim Anlauf**	Kap. 9

Die zweite Zeile ist die häufigste Lücke: Ein Vorgang, der aus drei Schritten besteht und nach dem zweiten abbricht, hinterlässt einen Zustand, den niemand vorgesehen hat. Entweder man macht ihn unteilbar, oder man erkennt und bereinigt ihn beim Anlauf.

Zwei Wege für den halben Vorgang.

	Unteilbar	Erkennen und bereinigen
Möglich bei	**einer Datenbank**	**über Nähte**
Aufwand	gering	hoch
Zwischenzustand sichtbar	nein	**ja**
Braucht	Transaktion	**Zustandsfeld**

Die erste Zeile ist die entscheidende Einschränkung: Eine Transaktion endet an der Datenbankgrenze. Über eine Naht hinweg gibt es keine — und der Versuch, eine zu bauen, ist die schwerste Aufgabe der verteilten Programmierung.

Die verteilte Transaktion

Eigenschaft	Wert	
Möglich	ja	**zweiphasig**
Verfügbarkeit	**schlechter als jeder Teil**	
Bei Ausfall des Koordinators	**alle blockiert**	**das Problem**
In der Praxis	selten	

Die dritte Zeile ist der Grund für die Seltenheit: Wenn der Koordinator zwischen den beiden Phasen ausfällt, halten alle Beteiligten ihre Sperren und warten — unbegrenzt. Ein Verfahren, das die Verfügbarkeit senkt, um Konsistenz zu sichern, ist selten der richtige Handel.

Der übliche Ausweg: Man verzichtet auf die Unteilbarkeit und arbeitet mit Ausgleichsschritten — was getan wurde, wird rückgängig gemacht. Das ist fachlich zu entwerfen, nicht technisch zu lösen.

Übungsaufgaben

(a) Entwerfen Sie den Wiederanlauf der Übertragung.

(b) Bestimmen Sie, was ein Ausgleichsschritt leistet.

(c) Prüfen Sie die Kette durch Versuche.

Lösung

(a) Wiederanlauf der Übertragung.

	Schritt	Dauer	Kritisch
1	Letzten Stand lesen	**40** ms	
2	**Offene Blöcke erkennen**	120 ms	**Zustands-feld**
3	Verbindungen aufbauen	2,1 s	1000 Geräte
4	**Puffer der Geräte leeren**	**34 s**	**Lastspitze**
5	Regelbetrieb	—	

Vierunddreißig Sekunden für Schritt vier, und dort liegt die Gefahr: Tausend Geräte senden gleichzeitig ihre Puffer — die zwanzigfache Regellast auf ein System, das gerade anläuft. Das ist der Ansturm aus Kapitel 9 in anderer Gestalt.

Maßnahme	Wirkung	
Geräte gestaffelt zulassen	**flache Last**	**Streuung**
Last begrenzen	ja	Band 22, Kap. 4
Erst danach Regelbetrieb	ja	
Anlaufdauer messen	**ja**	**Zielvorgabe**

Die letzte Zeile ist eine Betriebsanforderung, die selten formuliert wird: Wie lange darf ein Wiederanlauf dauern? Ohne diese Zahl kann niemand entscheiden, ob vierunddreißig Sekunden gut oder schlecht sind.

(b) Ausgleichsschritte.

Der Fall. Ein Befund wird gespeichert, die Warnung wird versendet, dann kommt eine Korrektur.

Schritt	Rückgängig?	
Befund speichern	**ja**	**Ersetzung eintragen**
Warnung erzeugen	ja	zurückziehen
Warnung angezeigt	**nein**	**gesehen ist gesehen**
Medikation angepasst	nein	fachliche Folge

Die dritte Zeile zeigt die Grenze jedes technischen Ausgleichs: Was ein Mensch gesehen hat, lässt sich nicht zurücknehmen. Der Ausgleich besteht darin, ihn zu benachrichtigen — und das ist eine fachliche Festlegung, keine technische.

Daraus folgt eine Entwurfsregel: Schritte mit unumkehrbaren Folgen kommen zuletzt. Erst speichern, prüfen, bestätigen — dann benachrichtigen. Wer die Reihenfolge umdreht, kann nicht mehr ausgleichen.

(c) Die Kette durch Versuche prüfen.

Versuch	Erwartet	Gefunden
Speicherung anhalten	**Puffer füllt sich**	richtig
Netz trennen	Wiederholung	richtig
Netz verlangsamen	Zeitschranke	**hing 30 s**
Halbe Antwort	Fehler	**Absturz**
Uhr verstellen	Ablehnung	gespeichert

Fünf Versuche, drei Befunde. Die ersten beiden Fälle sind die, an die jeder denkt — und sie funktionieren. Die letzten drei sind die, an die niemand denkt, und dort liegen die Fehler.

Der dritte Fall ist der lehrreichste: Ein langsames Netz ist schlimmer als ein getrenntes, weil die Zeitschranke greift statt eines sofortigen Fehlers. Genau das ist die Lage, die zur Kaskade aus Kapitel 10 führt.

Aufwand	Wert	
Versuchsaufbau	≈ 3 PT	**einmalig**
Je Versuch	≈ 20 min	
Gefundene Fehler	**3**	**davon 2 schwer**
Im Betrieb gefunden	—	während einer Störung

Drei Personentage gegen zwei schwere Fehler, die sonst während einer Störung gefunden würden — also zu dem Zeitpunkt, an dem sie am teuersten sind. Das ist dieselbe Rechnung wie bei den Prüfungen in Band 20, Kapitel 9.

Der Wiederanlauf wird nie geübt

Vorgang	Geübt?	
Neustart eines Dienstes	ständig	funktioniert
Wiederanlauf nach Ausfall	selten	
Rücksicherung	**fast nie**	**funktioniert oft nicht**
Ausweichbetrieb	nie	

Die dritte Zeile ist ein wiederkehrender Befund: Sicherungen werden angelegt und nie zurückgespielt. Erst im Ernstfall stellt sich heraus, dass sie unvollständig sind, die Rücksicherung sechs Stunden dauert oder ein Schlüssel fehlt.

Eine Sicherung, die nie zurückgespielt wurde, ist keine Sicherung, sondern eine Hoffnung. Die Übung gehört in den Betriebsplan — einmal im Quartal, mit gemessener Dauer.

Nygard[18] behandelt Ausfall und Wiederanlauf als Frage des Entwurfs; Bernstein und Newcomer[2] erklären die Verfahren zur Unteilbarkeit; Tanenbaum und van Steen[22] ordnen sie theoretisch ein.

Teil V

Das Urteil

Kapitel 13

Der zweite Entwurf im Großen

Was hier zusätzlich zählt

Kriterium	Band 20	Hier
Verstehenszeit	ja	ja
Änderbarkeit	ja	ja
Prüfbarkeit	ja	ja
Größenordnung	ja	ja
Übersteht Teilausfall	—	**ja**
Im Betrieb befragbar	—	**ja**
Einzeln erneuerbar	—	**ja**

Sieben Bedingungen, alle prüfbar — und die drei unteren lassen sich nicht nachträglich herstellen. Sie folgen aus der Bauform, und wer sie beim ersten Entwurf übergeht, baut beim zweiten das Werk um, nicht den Code.

Der Fall: die Verlaufsansicht durch das Werk

Der erste Entwurf.

Schwäche	Kapitel	Wirkung
Nur ein Zeitpunkt	4	**Verlauf falsch**
Einheit implizit	6	**Faktor** 18
n+1 über die Naht	8	5,1 s
Keine Zeitschranken	10	**Kaskade**
Fehler verschwiegen	10	**Lücke unsichtbar**
Keine Verfolgung	11	nicht aufklärbar
Wiederholung ohne Kennung	2	Werte doppelt

Sieben Schwächen, und jede stammt aus einem anderen Kapitel. Keine davon ist ein Codefehler — jede ist eine Entscheidung, die an einer Naht nicht getroffen wurde. Der Code selbst kann in jedem Bestandteil vorbildlich sein.

Der zweite Entwurf.

	Erster	Zweiter	
Zeilen gesamt	4200	**5100**	+21 %
Felder je Befund	6	**11**	
95. Perzentil	410 ms	**95** ms	4,3×
Bei Langzeitpatient	**5,1** s	**298** ms	17×
Teilausfall verkraftet	nein	**ja**	
Aufklärbar	nein	ja	
Schwächen	**7**	**0**	

Einundzwanzig Prozent mehr Zeilen, fünf zusätzliche Felder je Befund — und sieben Fehlerklassen fort. Das Muster aus Band 20 wiederholt sich: Der zweite Entwurf ist länger.

Was die fünf Felder kosten

Größe	Zuwachs	
Byte je Befund	16 → **34**	+112 %
Bei 10^9 Befunden	**+18 GB**	tragbar
Übertragungsdauer	+3 %	unerheblich
Beantwortbare Fragen	+5	**der Gewinn**

Achtzehn Gigabyte für eine Milliarde Befunde — bei heutigen Preisen ein zweistelliger Eurobetrag im Jahr. Dagegen stehen fünf Fragen, die vorher unbeantwortbar waren: Welcher Zeitpunkt? Welche Einheit? Wer hat wann was gesehen? Wurde korrigiert? Ist der Wert vollständig?

Übungsaufgaben

(a) Prüfen Sie die sieben Bedingungen.

(b) Bestimmen Sie die Reihenfolge des Umbaus.

(c) Untersuchen Sie, was nicht mehr zu ändern war.

Lösung

(a) Die sieben Bedingungen.

Bedingung	Prüfung	Ergebnis
Verstehenszeit	5 Fragen	88 s
Änderbarkeit	neues Feld	ohne Bruch
Prüfbarkeit	je Bestandteil	**einzeln**
Größenordnung	1000 Geräte	495 k/s
Teilausfall	**fünf Versuche**	**bestanden**
Befragbar	Verfolgung	ja
Einzeln erneuerbar	**Rollentausch**	ja

Die erste Zeile ist wieder der Preis: achtundachtzig Sekunden gegen einundsechzig beim ersten Entwurf. Ein Werk mit Zeitschranken, Wiederholungen und Verfolgung ist schwerer zu lesen als eines ohne — so wie in Band 21 und 22.

Die Antwort ist dieselbe wie dort und keine Rücknahme: Zerlegen und einen Kommentar an die Naht, der das Zusammenspiel erklärt. Danach liegt die Verstehenszeit bei zweiundsechzig Sekunden.

(b) Die Reihenfolge des Umbaus.

	Schritt	Aufwand	Wirkung
1	**Verfolgung einbauen**	4 PT	**sehen, was ist**
2	Zeitschranken	2 PT	keine Kaskaden
3	**Vertrag erweitern**	6 PT	**Zeit, Einheit**
4	Beide Seiten umstellen	14 PT	**Kap. 3**
5	Aufrufe bündeln	2 PT	17×
6	Fehler sichtbar machen	3 PT	

Schritt eins steht vorn, obwohl er nichts verbessert: Ohne Verfolgung weiß man nicht, ob die späteren Schritte wirken. Das ist dieselbe Regel wie in Band 21 und 22 — messen vor ändern.

Schritt vier ist der teuerste und der langwierigste: fünf Schritte über Wochen, weil beide Seiten nicht gleichzeitig umgestellt werden können. Das ist der Preis der Naht, den Kapitel 3 beziffert hat.

(c) Was nicht mehr zu ändern war.

Gegenstand	Änderbar?	
Code eines Bestandteils	leicht	
Naht zwischen zweien	mühsam	**Wochen**
Format nach außen	**kaum**	**zwölf Fremdsysteme**
Bedeutung eines Feldes	nie	**Altbestand**
Kennungsschema	nie	

Die vierte Zeile ist die härteste Grenze des Umbaus: Der gespeicherte Zeitpunkt war über Jahre der Speicherzeitpunkt. Man kann das Feld künftig richtig füllen — die Millionen alten Werte bleiben, wie sie sind, und der Verlauf bleibt für sie falsch.

Umgang mit Altbestand	Tauglich?	
Nachträglich ableiten	**nein**	Information fehlt
Schätzen	**nie**	**erfundene Daten**
Als unbekannt kennzeichnen	**ja**	**ehrlich**
Stichtag einführen	ja	**ab da verlässlich**

Die zweite Zeile ist die Versuchung, der man widerstehen muss: Aus dem Speicherzeitpunkt eine Abnahmezeit zu schätzen erzeugt Daten, die richtig aussehen und es nicht sind. Die dritte Zeile ist unbequem und die einzige zulässige.

Die Naht überlebt alles

Gegenstand	Lebensdauer
Eine Zeile Code	≈ 2 Jahre
Ein Bestandteil	≈ 7 Jahre
Eine Sprache im Werk	≈ 12 Jahre
Ein Datenformat	**≈ 20 Jahre**
Ein Kennungsschema	**unbegrenzt** **Patientendaten**

Die letzte Zeile ist die längste Verpflichtung, die man in einem Klinikvorhaben eingeht: Eine Patientenkennung muss die Aufbewahrungsfristen überdauern — dreißig Jahre und mehr. Sie wird jeden Bestandteil, jede Sprache und jeden Beteiligten überleben.

Deshalb rechtfertigt eine Naht einen Aufwand, der für Code unangemessen wäre. Zwei Wochen für ein Format, das zwanzig Jahre gilt, sind gut angelegt — und zwei Tage sind es nicht.

Fowler[8] liefert die Schritte des Umbaus im Großen; Ford und Richards[9] ordnen die Entscheidungen ein; Newman[17] behandelt die schrittweise Veränderung gewachsener Werke.

Kapitel 14

Wann teilen und wann nicht

Die vier Gründe, die tragen

Grund	Trägt?	
Andere Betriebsanforderung	**ja**	**Zeitschranke, Speicher**
Andere Sprache nötig	**ja**	Band 21, Kap. 14
Eigenständig erneuerbar	**ja**	
Eigenes Team, eigener Takt	**ja**	Conway

Die ersten beiden sind technisch zwingend — der Messdatenerfasser muss ein eigener Prozess in einer anderen Sprache sein. Die letzten beiden sind organisatorisch und deshalb ehrlicher, als sie klingen: Zuschnitt nach Zuständigkeit ist ein legitimer Grund, wenn man ihn benennt.

Die Gründe, die nicht tragen.

Behauptung	Trägt?	
“Besser skalierbar”	**selten**	**erst messen**
“Sauberer getrennt”	**nein**	**Module genügen**
“Leichter zu prüfen”	nein	schwerer
“Modern”	nein	

Die zweite Zeile ist die häufigste und die schwächste: Wer eine saubere Trennung will, führt ein Modul ein — Band 17, Kapitel 4. Ein eigener Prozess erzwingt die Trennung, und das ist der einzige Unterschied. Er kostet achtzehn Personentage.

Was ein zusätzlicher Bestandteil kostet

Posten	Aufwand	
Naht entwerfen und bauen	≈ 18 PT	**Kap. 1**
Bereitstellung, Überwachung	≈ 6 PT	laufend
Ausfallverhalten	≈ 5 PT	
Prüfung über die Naht	≈ 4 PT	**je Änderung**
Verfügbarkeit sinkt	—	**multiplikativ**
Verzögerung steigt	+3–80 ms	

Dreiunddreißig Personentage und eine schlechtere Verfügbarkeit — das ist der Einstandspreis. Er lohnt, wenn einer der vier Gründe vorliegt, und nicht, wenn man nur eine Grenze ziehen will.

Der Zuschnitt

Geschnitten nach	Folge	
Technischer Schicht	**jede Änderung trifft alle**	**schlecht**
Fachlichem Bereich	**Änderung bleibt lokal**	gut
Betriebs-anforderung	ja	zwingend
Abteilungsgrenze	ehrlich benennen	**Conway**

Die erste Zeile ist der klassische Fehlschnitt: ein Dienst für die Oberfläche, einer für die Logik, einer für die Daten. Eine neue Befundart ändert dann alle drei — und jede Änderung braucht drei abgestimmte Bereitstellungen.

Die Probe.

Typische Änderung	Trifft	
Neue Befundart	**1**	**guter Schnitt**
Neue Warnungsregel	1	gut
Neues Feld im Befund	**3**	unvermeidlich
Andere Anzeige	1	gut

Die Probe ist einfach und wird selten gemacht: Man nimmt die zehn letzten Änderungen und zählt, wie viele Bestandteile jede berührt hat. Liegt der Schnitt richtig, ist der Wert meist eins.

Der Weg zurück

	Teilen	Zusammenlegen
Aufwand	≈ 33 PT	**≈ 8 PT**
Risiko	mittel	gering
Wird gemacht	oft	**fast nie**

Zusammenlegen ist viermal billiger als Teilen und wird trotzdem kaum je erwogen — weil es wie ein Rückschritt aussieht. Dabei ist es die richtige Antwort, sobald zwei Bestandteile immer gemeinsam geändert und gemeinsam bereitgestellt werden.

Die Prüffrage lautet: Wurden diese beiden in den letzten zwölf Monaten je einzeln bereitgestellt? Lautet die Antwort nein, sind sie ein Bestandteil mit einer überflüssigen Naht dazwischen.

Übungsaufgaben

(a) Bewerten Sie das Vorhaben.

(b) Rechnen Sie den Einstiegsfall.

(c) Bestimmen Sie, womit man anfängt.

Lösung

(a) Das Vorhaben bewerten.

Bestandteil	Grund	Art	Urteil
Erfassung	Zeitschranke	**technisch**	**ja**
Übertragung	**Verfügbarkeit**	technisch	ja
Speicherung	Standard-system	technisch	ja
Auswertung	Fachlogik	**fachlich**	ja
Oberfläche	**eigener Takt**	organisat.	ja
Verlaufsdienst	“sauberer”	keiner	**nein**

Fünf Bestandteile mit tragenden Gründen und einer ohne. Der letzte ist der Vorschlag aus Kapitel 1 — er hätte zwei zusätzliche Nähte, dreiunddreißig Personentage und eine schlechtere Verfügbarkeit gekostet, für einen Teil, der sich nie unabhängig ändert.

(b) Der Einstiegsfall.
Die Lage. Ein kleineres Haus, ein Team von vier Personen.

	Fünf Teile	Zwei Teile
Erstellung	**34** PM	**21** PM
Nähte	4	**1**
Verfügbarkeit	99,3 %	**99,8** %
Bereitstellungen	5	2
Betriebsaufwand	≈ 0,8 Stellen	≈ **0,3**

Bei vier Personen sind zwei Bestandteile richtig: der Messdatenerfasser, weil die Zeitschranke es erzwingt, und alles andere in einem Programm. Dreizehn Personenmonate weniger und bessere Verfügbarkeit.

Die Trennung nach Fachbereichen bleibt — als Module innerhalb eines Programms, mit denselben Regeln aus Band 17. Wenn ein Teil später eigene Betriebsanforderungen bekommt, lässt er sich herauslösen. Der umgekehrte Weg ist teurer.

(c) Womit man anfängt.

1	**So wenige Bestandteile wie möglich**	**ein Programm, gut zerlegt**
2	Herauslösen, was muss	Zeitschranke, Sprache
3	**Messen, wo es klemmt**	Kap. 7
4	**Herauslösen, was sich belegen lässt**	
5	Zusammenlegen, was nicht trägt	**regelmäßig prüfen**

Der erste Schritt ist der unbeliebteste und der richtige: mit einem gut zerlegten Programm beginnen und nur herauslösen, was sich begründen lässt. Der umgekehrte Weg — viele Bestandteile von Anfang an — legt Nähte fest, bevor man weiß, wo sie hingehören.

Der fünfte Schritt fehlt in fast allen Vorhaben. Er kostet einen halben Tag im Quartal: die Änderungen der letzten Monate ansehen und fragen, welche Bestandteile immer gemeinsam betroffen waren.

Die Zahl der Nähte wächst schneller als die Zahl der Teile

Teile	Nähte	Aufwand	Verfügbar
2	**1**	33 PT	99,8 %
5	4	132 PT	99,3 %
12	**18**	594 PT	97,9 %
30	**60**	1980 PT	**94,9** %

Bei dreißig Bestandteilen ist die Verfügbarkeit auf knapp fünfundneunzig Prozent gefallen — achtzehn Tage Ausfall im Jahr, ohne dass ein einziger Bestandteil schlecht wäre. Das ist die Rechnung aus Kapitel 7, hier als Entscheidungsgrundlage.

Wer viele Bestandteile will, muss jeden einzelnen verfügbarer machen — und das kostet je Neun etwa das Dreifache. Der Aufwand wächst damit doppelt: mehr Nähte und höhere Anforderungen an jedes Teil.

Newman[17] behandelt Zuschnitt und Größe verteilter Dienste; Conway[6] erklärt, warum der Schnitt den Zuständigkeiten folgt; Richardson[21] sammelt die Bauformen mit ihren Kosten.

Kapitel 15

Der Stand im Jahr 2026

Was gilt und was behauptet wird

Aussage	Beleglage	Urteil
Fehler sitzen an den Nähten	gut	≈ 45 %
Der Mittelwert taugt nicht	eindeutig	Kap. 7
Idempotenz ist unverzichtbar	eindeutig	
Verfügbarkeit multipliziert sich	**Rechnung**	**Kap. 14**
Beobachtbarkeit gehört zum Entwurf	gut	
“Aufteilen macht sauberer”	**falsch**	**Module genügen**
“Verteilt ist skalierbar”	unvollständig	erst messen
“Ein Format für alles”	falsch	**eines je Naht**

Die vierte Zeile ist keine Erfahrung, sondern eine Rechnung — und deshalb die belastbarste Aussage des Bandes. Wer die Zahl der Bestandteile erhöht, senkt die Verfügbarkeit, es sei denn, er erhöht die jedes einzelnen.

Fünf Entwicklungen

Erstens: Die Pendelbewegung.

Zeitraum	Vorherrschend	
2005	Große Einzelanwendung	
2015	**möglichst viele Dienste**	Übertreibung
2022	Rückbau	gemessene Kosten
2026	**wenige, begründete**	**nach Anforderung**

Bemerkenswert ist die dritte Zeile: Mehrere große Anbieter haben Dienste wieder zusammengelegt und die Kosten offengelegt. Das hat die Diskussion versachlicht — die Frage lautet nicht mehr "wie viele Dienste", sondern "welcher Grund".

Zweitens: Fachstandards setzen sich durch.

Bereich	Stand	
Befundaustausch	**etabliert**	**FHIR**
Codelisten	etabliert	LOINC, UCUM
Verordnung	in Umsetzung	EU
Hausinterne Nähte	eigen	zulässig

Die letzte Zeile ist die praktische Empfehlung aus Kapitel 3: Fachstandard nach außen, eigenes Format innen, Übersetzung an einer Stelle. Das ist inzwischen die verbreitete Bauform und regulatorisch anerkannt.

Drittens: Beobachtbarkeit ist Standard geworden.

Praxis	Verbreitung	
Zählwerte	**üblich**	
Verfolgung über Nähte	verbreitet	**einheitliches Format**
Strukturierte Protokolle	üblich	
Ziele schriftlich	seltener	**die Lücke**

Die letzte Zeile ist der verbleibende Mangel: Die Werkzeuge sind da, die Zahlen werden erhoben — und niemand hat festgelegt, welcher Wert gut genug ist. Ohne Zielvorgabe ist ein Schaubild nur ein Schaubild.

Viertens: Regulierung erreicht die Nähte.

Anforderung	Betrifft	
Nachvollziehbarkeit	**beide Zeitachsen**	Kap. 4
Stückliste	alle Bestandteile	
Vollständigkeit belegen	**Zählwerte**	**Kap. 11**
Löschfristen	**auch Zwischenspeicher**	Kap. 9

Die dritte Zeile ist neu und folgenreich: Es genügt nicht, dass Befunde ankommen — man muss belegen können, dass keiner fehlt. Das verlangt einen Abgleich der Zählwerte über die ganze Kette, und den hat kaum jemand.

Fünftens: Werkzeuge, die Nähte erzeugen.

Wirkung	Befund	
Gerüst und Umsetzung	**gut**	**aus dem Schema**
Fehlerbehandlung	brauchbar	
Vertragsfragen	**unbeantwortet**	**Einheit, Zeitpunkt**
Idempotenz	selten	muss man verlangen

Die dritte Zeile ist der Kern: Ein Werkzeug erzeugt aus einem Schema tadellosen Code — und das Schema enthält genau die sieben Punkte aus Kapitel 3 nicht, die den Fall aus dem Vorwort verursacht haben. Es kann nur umsetzen, was jemand entschieden hat.

Drei Einsichten

Erstens: Die Naht ist die teuerste Entscheidung.

Gegenstand	Lebensdauer	Aufwand üblich
Code	≈ 2 Jahre	hoch
Bestandteil	≈ 7 Jahre	hoch
Naht	**≈ 20 Jahre**	**gering**
Kennungs-schema	**unbegrenzt**	**sehr gering**

Das Verhältnis ist umgekehrt proportional: Je länger etwas lebt, desto weniger Sorgfalt bekommt es. Das ist der Befund, aus dem dieser Band entstanden ist.

Zweitens: Fast die Hälfte der Fehler ist nicht Code.

Fehlerklasse	Ursache	Kapitel
Falscher Zeitpunkt	**Vereinbarung**	4
Falsche Einheit	Vereinbarung	6
Werte doppelt	**fehlende Kennung**	2
Lücke unsichtbar	**Anzeige-entscheidung**	10
Kaskade	fehlende Schranke	10
Veralteter Wert	Zwischen-speicher	9

Sechs Fehlerklassen, keine davon ein Programmierfehler. Jede entsteht dort, wo zwei richtige Programme sich über etwas nicht geeinigt haben — und keine wird von einem Werkzeug gefunden, das ein Programm prüft.

Drittens: Das Handwerk trägt auch hier.

Aus Band 20	Gilt hier?	
Aufgabe verstehen	**unverändert**	
Daten zuerst	**verstärkt**	**der Vertrag**
Fehlersuche als Verfahren	unverändert	**plus Verfolgung**
Fremden Code lesen	unverändert	
Wann aufhören	plus drei Kriterien	

Viermal dasselbe Ergebnis über vier Bände. Die zweite Zeile fällt hier am stärksten aus: Der Vertrag einer Naht ist ein Datenmodell, das zwanzig Jahre gilt — Wirths Rangordnung auf ihrer längsten Zeitskala.

Was dieser Band nicht behauptet

Behauptung	Aufgestellt?	
Verteilen ist schlecht	**nein**	**vier Gründe**
Es gibt eine richtige Zahl von Diensten	**nein**	
Nähte lassen sich fehlerfrei bauen	nein	
Diese Zahlen sind Konstanten	nein	**Größen-ordnungen**

Die zweite Zeile ist die wichtigste. Die richtige Zahl folgt aus den Anforderungen: Ein Vierpersonenteam braucht zwei Bestandteile, ein Universitätsklinikum vielleicht zwölf. Wer eine Zahl nennt, ohne die Anforderungen zu kennen, rät.

Ein Programm löst eine Aufgabe.
Ein Werk hält ein Versprechen —
über Jahre, über Ausfälle,
über alle hinweg, die daran gebaut haben.

— die Bilanz dieses Bandes.
Das Versprechen steht in den Nähten.
Nicht im Code.

Kleppmann[15] ordnet die Entscheidungen verteilter Werke; Brewer[4] liefert den Rahmen für die Abwägung; Abbott und Fisher[1] behandeln das Wachstum solcher Werke über Jahre.

Teil VI

Anhang

Kapitel A

Regeln und Kenngrößen

Die Regeln

Zerlegung	Erst schneiden, dann verteilen — und nur mit Grund
Naht	Grobe Aufrufe, Mengen statt Einzelstücke
Idempotenz	**Kennung vom Sender, sonst keine Wiederholung**
Vertrag	Einheit, Genauigkeit, Bedeutung des Fehlens
Zeit	**Beide Achsen führen; intern UTC, außen Ortszeit**
Text	Einmal normalisieren, an der Eintrittsstelle
Zahlen	**Rundung zweimal: am Anfang und am Ende**
Prüfung	Beide Seiten prüfen; Plausibilität an der Naht
Messen	Perzentile, nicht Mittelwerte
Fehler	**Zeitschranken überall, Fristen weiterreichen**
Anzeige	Unvollständigkeit immer sichtbar machen
Aufzeichnung	Beim Entwurf einbauen, nicht danach

Die wichtigsten Zahlen

Größe	Wert	Kapitel
Fehler an den Nähten	$\approx 45\,\%$	1
Aufruf gegen Netzaufruf	Faktor $\approx 250\,000$	1
Aufwand je Naht	≈ 18 PT	1
Einstand je Bestandteil	≈ 33 PT	14
Nähte bei n Teilen	bis $n(n-1)/2$	14
$n+1$ über die Naht	Faktor ≈ 65	8
JSON gegen binär	Faktor ≈ 9	2
Zeit in Datenbankabfragen	$\approx 48\,\%$	8
20 Dienste, je 1 % langsam	$\approx 18\,\%$	7
Verfügbarkeit, 4 Teile	$\approx 99{,}3\,\%$	7
Je Neun	etwa $3\times$ teurer	7
Glukose mg/dl zu mmol/l	Faktor ≈ 18	6
Uhrabweichung, eingebettet	5–30 s/Tag	4
Aufzeichnung beim Entwurf	$\approx 3\,\%$	11
Aufzeichnung danach	$\approx 12\,\%$	11

Alle Werte sind Größenordnungen. Die Verhältnisse sind belastbarer als die absoluten Zahlen — und auf Verhältnissen beruhen die Argumente dieses Bandes.

Der Nahtvertrag als Prüfliste

1	Felder, Typen, Pflicht oder optional
2	**Einheit — und wie sie geprüft wird**
3	**Genauigkeit — Betrag und Exponent, nicht Gleitkomma**
4	Wertebereich: möglich, kritisch, Referenz
5	**Bedeutung des Fehlens — ausdrücklich**
6	**Welcher Zeitpunkt — und welche Zone**
7	Kodierung und Normalform
8	**Kennung: wer vergibt sie, woraus besteht sie**
9	Verhalten bei Wiederholung
10	**Verhalten bei teilweise ungültigen Mengen**
11	Zeitschranke und wer sie weitergibt
12	**Wie das Format erweitert wird**

Zwölf Punkte. Die Punkte 1 und 4 stehen in jedem Schema; die übrigen zehn stehen fast nirgends — und aus ihnen stammen die Fehlerklassen dieses Bandes.

Kapitel B

Landkarte und Glossar

Wo was steht

Kap.	Thema	Verweise
1	Programm und Werk	**Band 17, Kap. 4**
2	Die Naht	**Band 21, Kap. 14**
3	Verträge	**Band 17, Kap. 6**
4	Zeit	**Band 20, Kap. 2**
5	Text	**Band 21, Kap. 4**
6	Zahlen	**Band 21, Kap. 8**
7	Messen im Verbund	Band 21, Kap. 12
8	Wo die Zeit hingeht	**Band 20, Kap. 12**
9	Zwischenspeicher	**Band 22, Kap. 12**
10	Fehler im Verbund	**Band 22, Kap. 2**
11	Beobachtbarkeit	**Band 18, Band 19**
12	Ausfall	**Band 12**
13	Der zweite Entwurf	**Band 20, Kap. 13**
14	Wann teilen	**Band 17, Kap. 4**
15	Stand 2026	

Begriffe

Ausgleichsschritt	Fachliche Rücknahme statt technischer Unteilbarkeit
Fachliche Zeit	Wann etwas geschah
Gegendruck	Bremsen des Senders bei Überlast
Idempotenz	Zweimal wirkt wie einmal
Naht	Vereinbarte Grenze zwischen zwei Bestandteilen
Perzentil	Wert, unter dem ein Anteil der Messungen liegt
Sättigung	Auslastung als Vorwarngröße
Sicherungsschalter	Aufrufe abweisen, solange der Empfänger ausfällt
Sonderablage	Ort für endgültig fehlgeschlagene Nachrichten
Systemzeit	Wann wir es wussten
Teilausfall	Ein Bestandteil fällt aus, andere laufen
Verfolgung	Eine Kennung wandert durch alle Bestandteile
Werk	Zusammengesetztes System aus mehreren Bestandteilen

Kapitel C

Kommentierte Quellen

[1] Abbott, M., Fisher, M.: *The Art of Scalability*. 2. Aufl., Addison-Wesley 2015.

Behandelt Wachstum von Werken über Jahre — technisch und organisatorisch. Die Verbindung beider Seiten ist selten und für Kapitel 14 unmittelbar brauchbar.

[2] Bernstein, P., Newcomer, E.: *Principles of Transaction Processing*. 2. Aufl., Morgan Kaufmann 2009.

Erklärt die Verfahren zur Unteilbarkeit, einschließlich der zweiphasigen Freigabe und ihrer Grenzen. Trocken, gründlich und die maßgebliche Quelle zu Kapitel 12.

[3] Beyer, B., u. a.: *Site Reliability Engineering*. O'Reilly 2016.

Das Buch, das Betrieb zur Entwurfsfrage macht. *Zielvorgaben, Fehlerbudgets, Alarme mit Handlungsbezug — und ein ehrlicher Bericht darüber, was in großem Maßstab schiefgeht. Frei verfügbar.*

[4] Brewer, E.: *CAP Twelve Years Later*. IEEE Computer 45(2), 2012.

Der Rückblick des Urhebers auf ein oft verkürzt zitiertes Ergebnis. Räumt mit der Vorstellung auf, man müsse zwei von drei Eigenschaften wählen — die Abwägung ist feiner und liegt bei jeder Operation neu.

[5] Brooks, F.: *The Mythical Man-Month*. Jubiläumsausgabe, Addison-Wesley 1995.

Über Kommunikationsaufwand in Vorhaben. *Für diesen Band unmittelbar anwendbar: Jede Naht ist Kommunikation, in Code gegossen — und wächst quadratisch mit der Zahl der Teile.*

[6] Conway, M.: *How Do Committees Invent?* Datamation 14(4), 1968.

Fünf Seiten, die eine bis heute gültige Beobachtung begründen: Ein System bildet die Kommunikationsstruktur seiner Erbauer ab. In jedem größeren Vorhaben nachprüfbar.

[7] Evans, E.: *Domain-Driven Design*. Addison-Wesley 2003.

Begründet, warum Fachbegriffe und nicht Objekte über eine Naht gehen, und liefert den Begriff des abgegrenzten Kontexts. Umfangreich; die Kapitel zu Kontextgrenzen genügen für diesen Band.

[8] Fowler, M.: *Patterns of Enterprise Application Architecture*. Addison-Wesley 2002.

Sammelt die Bauformen für Datenzugriff und Verteilung — einschließlich der Muster gegen n+1. Alt und in der Sache unverändert gültig.

[9] Ford, N., Richards, M., u. a.: *Fundamentals of Software Architecture*. O'Reilly 2020.

Ordnet die Entwurfsentscheidungen im Großen und benennt zu jeder die Abwägung. Nüchtern und ohne Empfehlung einer bestimmten Bauform — das ist selten.

[10] Goldberg, D.: *What Every Computer Scientist Should Know About Floating-Point Arithmetic*. ACM Computing Surveys 23(1), 1991.

Der Aufsatz zum Thema. Hier zählt vor allem die Frage, was beim Übertragen einer Zahl verlorengeht. Anspruchsvoll und die Mühe wert.

[11] Gregg, B.: *Systems Performance*. 2. Aufl., Addison-Wesley 2020.

Das Standardwerk zur Leistungsmessung. *Liefert die Methodik — welche Kennzahl für welche Frage — und die Werkzeuge darunter. Für Kapitel 7 und 8 die Grundlage.*

[12] Hohpe, G., Woolf, B.: *Enterprise Integration Patterns*. Addison-Wesley 2003.

Der Katalog der Verbindungsbauformen. *Warteschlangen, Weiterleitung, Umwandlung, Wiederholung — mit Anlass und Gegenanzeige zu jedem. Alt, umfangreich, unersetzt.*

[13] IEEE 754: *Standard for Floating-Point Arithmetic*. 2019.

Legt Darstellung und Rundungsregeln fest. Zum Nachschlagen, wenn man wissen muss, was bei 0,5 *geschieht und warum zwei Systeme verschieden runden.*

[14] RFC 3339 und ISO 8601: *Date and Time Formats*.

Legen das Zeitformat fest, das über Nähte gehört — einschließlich Zonenversatz. Kurz und die verbindliche Quelle zu Kapitel 4.

[15] Kleppmann, M.: *Designing Data-Intensive Applications*. O'Reilly 2017.

Das beste Buch über verteilte Datenverarbeitung. *Ordnet Konsistenz, Replikation, Zeit und Fehler in einen Zusammenhang, ohne eine bestimmte Technik zu bewerben. Wenn Sie eines lesen, dann dieses.*

[16] Majors, C., Fong-Jones, L., Miranda, G.: *Observability Engineering*. O'Reilly 2022.

Entwickelt den Begriff der Beobachtbarkeit und grenzt ihn von Überwachung ab: nicht bekannte Fragen beantworten, sondern unbekannte stellen können. Für Kapitel 11 maßgeblich.

[17] Newman, S.: *Building Microservices*. 2. Aufl., O'Reilly 2021.

Behandelt Zuschnitt und Größe verteilter Dienste — und ist in der zweiten Auflage deutlich zurückhaltender als in der ersten. Die Kapitel zu Kosten und Rückbau sind die wertvollsten.

[18] Nygard, M.: *Release It!* 2. Aufl., Pragmatic Bookshelf 2018.

Das Buch über Ausfall. *Kaskaden, Sicherungsschalter, Zeitschranken — alles an echten Fällen entwickelt. Für Kapitel 10 und 12 die Grundlage und ausgesprochen unterhaltsam.*

[19] Parnas, D.: *On the Criteria To Be Used in Decomposing Systems into Modules*. CACM 15(12), 1972.

Fünf Seiten, die den Modulbegriff begründet haben. Hier die Erinnerung daran, dass die Zerlegungsfrage vor der Verteilungsfrage steht — und dass ein Modul oft genügt, wo ein Dienst erwogen wird.

[20] Postel, J.: *Robustness Principle*. RFC 793 u. a., 1981.

"Streng im Senden, nachsichtig im Empfangen" — die Regel, die Erweiterbarkeit ermöglicht. Heute mit Vorbehalt zu lesen: Zu viel Nachsicht beim Deuten hat Sicherheitslücken erzeugt.

[21] Richardson, C.: *Microservices Patterns*. Manning 2018.

Sammelt die Bauformen mit ihren Kosten — Ausgleichsschritte, Idempotenz, verteilte Abfragen. Konkreter als [17] und mit durchgearbeiteten Beispielen.

[22] Tanenbaum, A., van Steen, M.: *Distributed Systems*. 4. Aufl., 2023.

Das Lehrbuch zur Theorie — Uhren, Ordnung, Übereinstimmung, Fehlermodelle. Frei verfügbar. Wer wissen will, warum etwas nicht geht, findet hier den Beweis.

[23] The Unicode Consortium: *The Unicode Standard* und UAX #15.

Legt Normalformen und Vergleichsregeln fest. Umfangreich; für Kapitel 5 genügen die Abschnitte zur Normalisierung und zur Faltung.

[24] Vogels, W.: *Eventually Consistent*. CACM 52(1), 2009.

Beschreibt, was verzögerte Konsistenz praktisch bedeutet — aus der Sicht von jemandem, der ein solches System betrieben hat. Kurz und nüchtern.

[25] Waldo, J., u. a.: *A Note on Distributed Computing*. Sun Microsystems 1994.

Der Aufsatz, der die Gleichsetzung von Aufruf und entferntem Aufruf widerlegt. *Über dreißig Jahre alt und wird in jedem Jahrzehnt neu gebraucht, weil der Irrtum wiederkehrt.*

[26] Wirth, N.: *Algorithms + Data Structures = Programs*. Prentice Hall 1976.

Der Titel ist die These und enthält eine Rangordnung. Auf ein Werk angewandt: Der Vertrag einer Naht ist ein Datenmodell — und er lebt länger als jedes Programm, das ihn benutzt.

[27] HL7: *FHIR Specification*. Fortlaufend.

Ein ausgearbeiteter Fachvertrag samt Codelisten. *Für den Austausch klinischer Daten der Maßstab — umfangreich und deshalb in der Praxis über Profile eingeschränkt. Frei verfügbar.*

[28] Dean, J., Barroso, L.: *The Tail at Scale*. CACM 56(2), 2013.

Erklärt, warum Ausläufer sich über Dienstketten aufschaukeln *— und was man dagegen tut. Sechs Seiten, die die Perzentildiskussion aus Kapitel 7 begründen.*

Wenn Sie nur drei Bücher kaufen

[15]	**Kleppmann**	**Für das Ganze**
[18]	**Nygard**	Für den Ausfall
[11]	**Gregg**	**Für das Messen**

Kleppmann ordnet, Nygard warnt, Gregg misst. Auffällig ist, was fehlt: kein Buch über eine bestimmte Bauform verteilter Dienste — diese Bücher altern schneller als die Fragen, die sie beantworten.

Zwei Ergänzungen, beide kurz und frei verfügbar: [25] Waldo, acht Seiten gegen einen

> *Irrtum, der jedes Jahrzehnt zurückkehrt. Und [6] Conway — fünf Seiten, nach denen man Organigramme anders liest.*